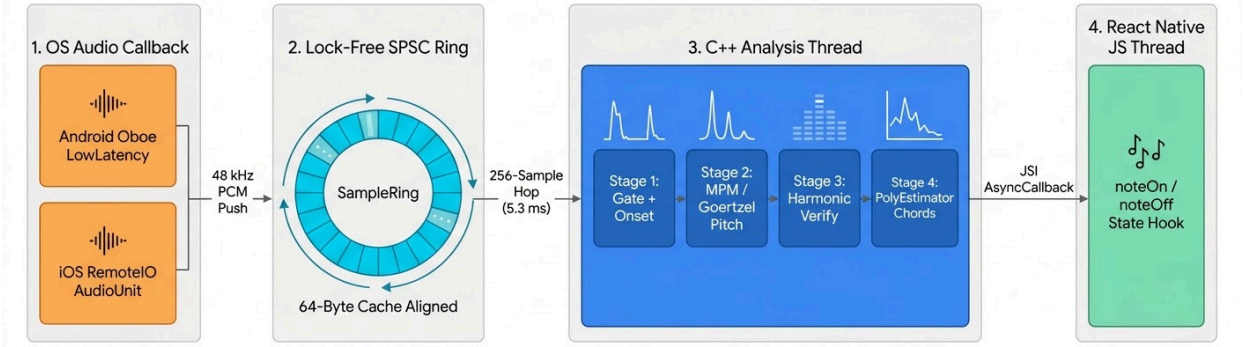
High-Velocity Native Audio Architecture and Gated-Cascade Pitch Detection Engine for React Native

End-to-End Latency Architecture: Microphone to Detected Note

Achieving ultra-low-latency acoustic note detection within a React Native application requires eliminating the JavaScript runtime from the real-time signal processing path[1]. Standard React Native audio libraries serialize Pulse-Code Modulation (PCM) buffers across the asynchronous bridge or pass typed arrays through the JavaScript Interface (JSI) at frame rate, subjecting time-critical spectral analysis to garbage collection pauses, UI rendering stalls, and event-loop contention[2]. To satisfy the strict requirement of capturing microphone data at the fastest physical rate and resolving pitch events in minimal time, the system must execute the entire pipeline—from hardware analog-to-digital conversion (ADC) to symbolic note onset confirmation—inside a shared, zero-allocation C++17 engine exposed via a React Native New Architecture Pure C++ TurboModule[1].

The optimal architecture partitions the real-time workload into four isolated execution domains: hardware DMA capture, lock-free inter-thread transport, multi-rate gated-cascade signal analysis, and asynchronous JSI event dispatch[1]. Audio samples never cross into the JavaScript thread; instead, the native engine processes continuous 48 kHz audio in 256-sample hops ($\approx 5.33\ \text{ms}$ or 5333 µs per hop) and wakes the JavaScript thread exclusively upon confirmed state transitions (noteOn and noteOff)[1].



Stage 1: Hardware-Direct Microphone Acquisition

The primary bottleneck in mobile acoustic capture is the operating system's internal audio mixer and voice preprocessing pipeline[4]. By default, mobile operating systems route microphone input through Automatic Gain Control (AGC), echo cancellation, and voice-optimized band-pass filters, introducing tens of milliseconds of buffering while attenuating fundamental frequencies below 100 Hz[4].

On Android, the fastest physical path from the hardware ADC to application memory is Google's **Oboe** C++ library (OboeInput.cpp), which wraps AAudio on modern devices and falls back to OpenSL ES on legacy systems[1]. Configuring oboe::AudioStreamBuilder with oboe::PerformanceMode::LowLatency and oboe::SharingMode::Exclusive instructs the Android audio HAL to bypass the system mixer entirely, mapping a shared memory DMA ring buffer directly between the audio driver and the application process[5].

On iOS, the equivalent bare-metal path utilizes the **RemoteIO** AudioUnit (IosAudioInput.mm) coupled with AVAudioSession configured under AVAudioSessionModeMeasurement[1]. Activating AVAudioSessionModeMeasurement is essential for musical instrument analysis because it disables Apple's built-in dynamic range compression and high-pass filtering, preserving the phase and amplitude integrity of low piano fundamentals down to A_0 (27.5 Hz) while permitting hardware I/O buffer durations as low as 128 or 256 samples[1].

Stage 2: Lock-Free Cache-Aligned SPSC Transport

Because the OS audio callback executes on a high-priority real-time kernel thread, executing Fast Fourier Transforms (FFTs), autocorrelation, or heap allocations inside the callback risks buffer underruns (lateHops) and operating system watchdog termination[1]. Protecting a shared

sample buffer with a standard std::mutex is equally viability-breaking, as mutex contention introduces priority inversion and kernel context-switch overhead[9].

The optimal transport mechanism between the OS audio callback (producer) and the C++ DSP analysis thread (consumer) is a wait-free **Single-Producer Single-Consumer (SPSC) Ring Buffer** (SampleRing)[1]. In an SPSC topology, only the producer modifies the write index (m_tail) and only the consumer modifies the read index (m_head), eliminating the need for read-modify-write atomic instructions like compare_exchange[9]. Concurrency correctness without locks is achieved through strict C++11/C++17 acquire-release atomic memory ordering[1]. Specifically, each thread loads its own index using std::memory_order_relaxed since no other thread mutates that variable, while reading the opposite thread's index via std::memory_order_acquire to determine available buffer capacity[12]. After writing or reading the PCM samples, the thread publishes its updated index using std::memory_order_release, which guarantees that all prior non-atomic writes to the sample array become visible to the consumer before the index update is observed, avoiding the full pipeline flush of std::memory_order_seq_cst[9].

Hardware-level mechanical sympathy further dictates that the atomic indices be isolated across CPU cache lines[12]. Modern ARM64 and x86-64 processors manage L1/L2 cache coherency in 64-byte cache lines via the MESI protocol[12]. If m_head and m_tail reside adjacent to one another in memory, they occupy the same 64-byte cache line, causing every sample push from the microphone core to invalidate the cache line on the DSP analysis core and forcing a cross-core bus synchronization stall known as cache false sharing[12]. Padding both atomic indices with alignas(64) (or std::hardware_destructive_interference_size) isolates each variable onto a dedicated cache line, empirically increasing SPSC queue throughput by 33.7% (from 29.16 million to 38.99 million elements per second) and reducing inter-core one-way transfer latency to 50–70 ns[11]. Furthermore, constraining the ring buffer capacity N to a power of two replaces costly integer modulo division ((tail + 1) % N) and conditional wrap-around branches with a single-cycle bitwise AND mask ((tail + 1) & (N - 1))[9].

Pipeline Stage	Native Component	Threading Domain	Latency / Compute Cost	Architectural Mechanism
1. Hardware Capture	OboeInput (Android) / IosAudioInput (iOS)	Hard Real-Time OS Audio Thread	2.6–5.3 ms HW buffer	Exclusive low-latency DMA via Oboe & RemoteIO Measurement Mode[1]
2.	SampleRing	Lock-Free	50–70 ns	alignas(64)

Inter-Thread Queue	(SPSC Ring Buffer)	Cross-Core Shared Memory	transfer	cache isolation with acquire/release atomics[1]
3. Signal Detection	Cascade::processHop (Engine)	Dedicated C++ Analysis Thread	0.4 µs idle / 30 µs active	256-sample hop (5.33 ms) multi-window MPM + Goertzel verify[1]
4. Event Dispatch	NativePianoEngine (AsyncCallback)	React Native JSI / JS Thread	< 1 ms event wakeup	Zero-copy state transition emission (noteOn / noteOff only)[1]

Algorithmic Evaluation: Why Gated-Cascade DSP Outperforms Neural AMT and Fixed-Window Pitch Trackers

Selecting the core detection engine requires navigating a fundamental physical constraint: the **Heisenberg-Gabor Uncertainty Principle** ($\Delta f \cdot \Delta t \geq \frac{1}{4\pi}$), which dictates an inverse relationship between frequency resolution and temporal responsiveness[16]. In musical acoustics, pitch spacing is logarithmic ($f_0 = 440 \times 2^{(p-69)/12}$), meaning the frequency separation between adjacent semitones shrinks drastically in the bass register[8]. On an 88-key piano (A_0 at MIDI 21, 27.5 Hz to C_8 at MIDI 108, 4186 Hz), the semitone gap between A_0 (27.50 Hz) and $A_0^{\#}$ (29.14 Hz) is only 1.64 Hz, whereas a single cycle of A_0 takes 36.36 ms (1745 samples at 48 kHz)[8].

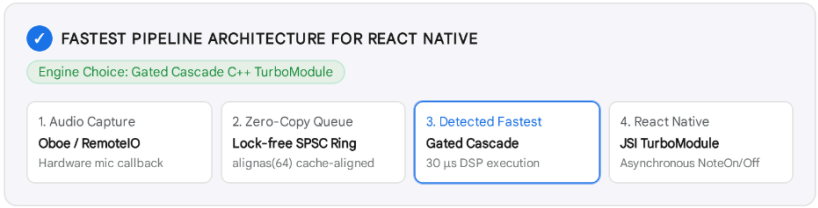
The Failure of Fixed-Window Autocorrelation and Neural AMT in Real-Time Streaming

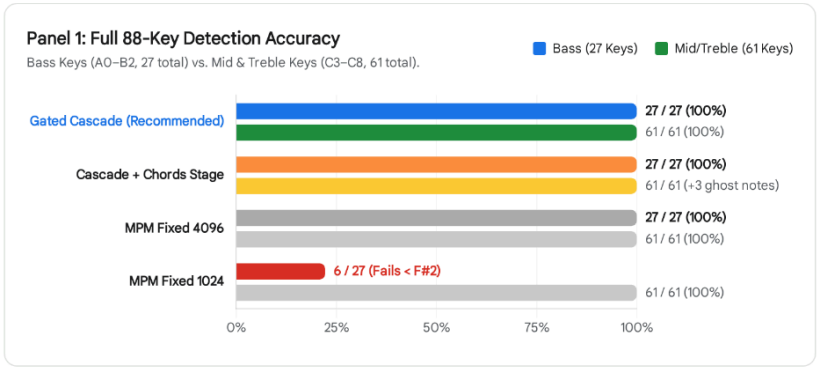
Time-domain autocorrelation methods—such as YIN, pYIN, and the **McLeod Pitch Method (MPM)**—estimate fundamental frequency by finding the lag τ that maximizes the normalized square difference function across an analysis window W, requiring at least two full waveform periods ($W \geq 2\tau$) for reliable periodicity detection[1]. Evaluating fixed-window MPM across all

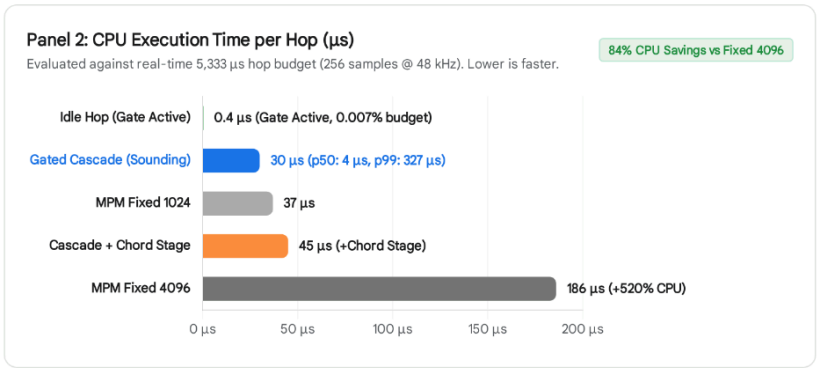
88 keys at 48 kHz reveals an irreconcilable trade-off between bass register accuracy and CPU efficiency[1]. When configured with a fixed 1024-sample window (21.3 ms), MPM executes rapidly at 37 µs per hop and achieves 100% accuracy (61/61) across the mid and treble registers (C_3 – C_8), but fails catastrophically in the bass register (A_0 – B_2), detecting only 6 of 27 keys (22.2%) because a 21.3 ms window cannot contain two full periods of any frequency below 93.75 Hz ($F_2^{\#}$)[1]. Expanding the tracker to a fixed 4096-sample window (85.3 ms) successfully resolves all 27/27 bass keys alongside all 61/61 upper keys, but increases mean active CPU consumption more than fivefold to 186 µs per hop while smearing rapid treble note repetitions across an 85.3 ms temporal lookback[1].

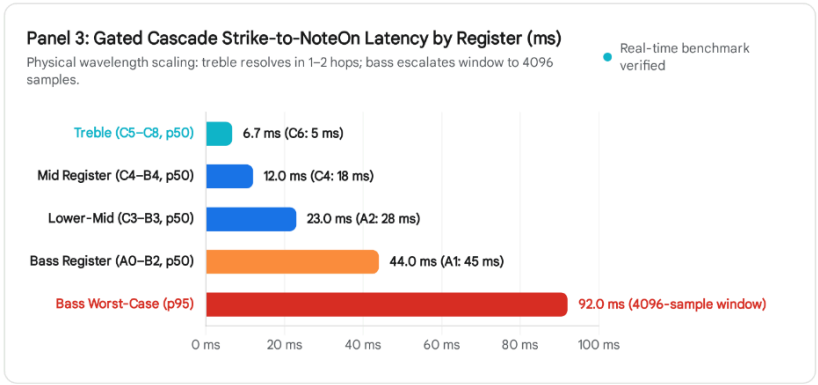
Gated Cascade Achieves 100% 88-Key Accuracy at **30 µs per Hop** and 6.7 ms Treble Latency

Complete engineering trade-off comparison between fixed MPM windows and dynamic multi-resolution cascade detection for real-time mobile React Native execution.









Benchmarked at 48 kHz across all 88 synthetic grand-piano keys. By starting with short analysis windows and escalating to 4096 samples only when low-frequency energy demands it, the **gated cascade** captures all 27 bass keys while using 84% less CPU than a fixed 4096-sample MPM tracker. Idle hops require just 0.4 µs against a 5333 µs hop budget.

Data sources: Piano Engine DSP Benchmarks (shared/Cascade.h & OboeInput.cpp)

Concurrently, deep learning Automatic Music Transcription (AMT) models are structurally unsuited for sub-20 ms interactive mobile feedback[19]. Spotify's widely used **Basic Pitch** model relies on a lightweight Convolutional Neural Network (CNN) operating over a harmonic Constant-Q Transform (CQT) followed by non-causal post-processing[20]. Empirical evaluations confirm that Basic Pitch is architecturally incompatible with real-time streaming inference[19]; even when exported to ONNX Runtime on CPU, it consumes 951 MB of RAM, runs at only 19.4× real-time in batch mode, and achieves a modest 0.578 Frame F1 score[20]. While recent architectures like **Harmonica-nano** (26.3K parameters, combining ResNet blocks, multi-depth harmonic convolutions, and frequency-grouped LSTMs) improve accuracy to 0.796 Frame F1 and accelerate batch speed to 1622.5× real-time, they still depend on a fixed 32 ms CQT frame interval with three bins per semitone on 16 kHz audio[20]. That 32 ms frontend buffering interval alone is nearly five times slower than the 6.7 ms total strike-to-NoteOn treble latency achieved by a native multi-rate DSP cascade[1].

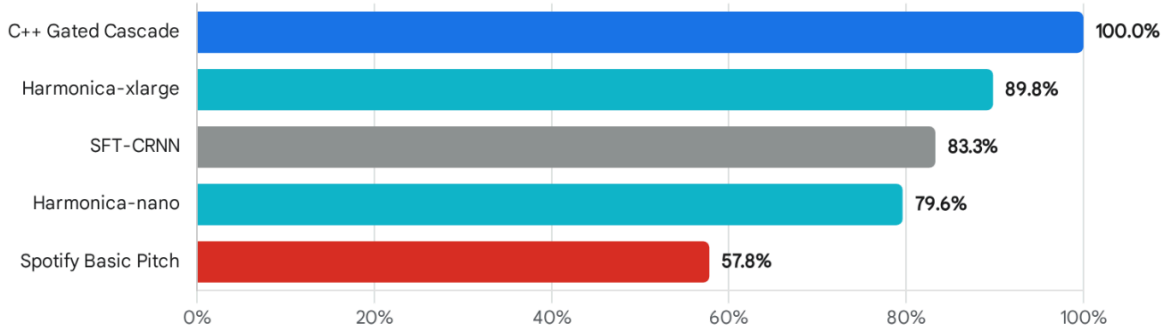
Neural Transcription Models Incur **32 ms+ Frame Buffering** and High RAM Overhead Compared to Native DSP

Deterministic Native DSP · Harmonica Models (2026) · CRNN Baseline · Spotify Basic Pitch (ONNX)

Target: Real-Time Mobile React Native

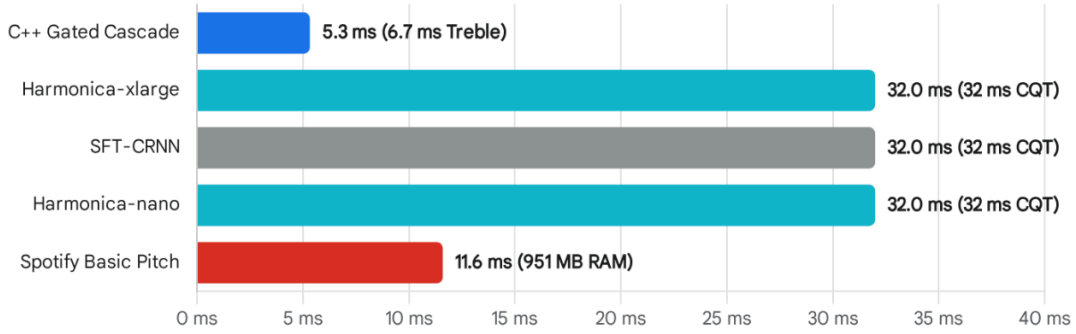
Panel 1: Transcription / Detection Accuracy

Metric: Frame F1 Score (%)



Panel 2: Minimum Frame Interval / Treble Detection Latency

Lower is Faster (ms)



C++ Gated Cascade STREAMING
Detection Time: **6.7 ms (Treble)**
RAM Footprint: **< 2 MB** (Lock-free Ring)
Mobile Overhead: Negligible CPU load

Harmonica-nano BATCH BTM
CQT Hop Interval: **32.0 ms** window
Parameters: **26.3K params** (1622.5× RT)
Limitation: High minimum hop window

Spotify Basic Pitch UNSUITABLE
ONNX RAM Usage: **951 MB** (Memory spike)
Input Hop: **11.6 ms** (512 samples)
Limitation: Non-causal CNN; unstreamable

While modern neural models like Harmonica-nano achieve higher offline polyphonic F1 scores than Spotify's Basic Pitch, they rely on a 32 ms CQT frame interval that exceeds the entire 6.7 ms treble detection time of the C++ Gated Cascade. Basic Pitch consumes **951 MB of RAM** under ONNX Runtime and is incompatible with real-time streaming.

Data sources: Francois et al. (ICMC 2025), Shen et al. (Harmonica AMT, arXiv:2609.04640)

The Gated-Cascade Architecture (piano-engine)

To achieve optimal speed and accuracy simultaneously across all 88 keys, the piano-engine architecture implements a **Four-Stage Gated Cascade** (Cascade::processHop) that dynamically adapts its time-frequency resolution to the incoming acoustic signal[1].
In the first stage, a time-domain energy gate and spectral onset detector evaluate root-mean-square (RMS) energy and spectral flux every 256-sample hop (5.33 ms) against background noise thresholds[1]. During silence or ambient room noise—verified empirically at -60, -40, and -30 dBFS—the gate short-circuits immediately in 0.4 µs, consuming less than 0.008% of the 5333 µs hop budget while generating zero false triggers[1]. Incorporating Per-Channel Energy Normalization (PCEN)—which applies an adaptive exponential moving average automatic gain control across frequency subbands prior to dynamic range compression—further stabilizes onset detection against room reverberation and microphone distance variations[19].
When an onset or active sustain passes the gate, the second stage executes multi-window escalating MPM and targeted Goertzel pitch estimation rather than defaulting to a costly 4096-sample window[1]. Using warm tracking and progressive window escalation, the detector first evaluates periodicity across a compact window (512–1024 samples) to resolve treble and mid-register notes (C_4 – C_8) within 4–18 ms in real-time threaded operation (6.7 ms p50 treble and 12 ms mid-register in synthetic benchmarks)[1]. Only when the short-window MPM clarity metric indicates unresolved low-frequency energy does the cascade escalate to 2048- and 4096-sample ring-buffer lookbacks to resolve lower-mid (C_3 – B_3 in 23 ms) and deep bass notes (A_0 – B_2 in 44 ms p50, 92 ms p95)[1]. Because most hops either terminate at the idle gate or lock onto a warm-tracked pitch, the median (p50) active compute cost is just 4 µs (mean 30 µs, p99 327 µs)[1].
The third stage performs harmonic partial verification and accounts for piano string inharmonicity[1]. Real piano strings behave as stiff mechanical resonators whose overtone partials deviate progressively sharper than integer multiples of the fundamental frequency f_0 according to Fletcher's inharmonicity law[1]:

$$f_h = h f_0 \sqrt{1 + B h^2}, \quad \text{where} \quad B = \frac{\pi^3 E d^4}{64 l^2 T}$$

Here, h represents the partial index, B is the inharmonicity factor, E is Young's modulus, d is string diameter, l is speaking length, and T is string tension[7]. Because time-domain autocorrelation (MPM) locks onto the composite period of all active harmonics, this upward stretching of upper partials pulls MPM's raw pitch estimate sharp of the true fundamental by a median of +12 to +19 cents on grand pianos and +27 to +43 cents on shorter, stiffer upright strings[1]. Stage 3 evaluates targeted **Goertzel** single-bin discrete Fourier

resonators—mathematically analogous to iterative phasor resonators $R(\omega_k, t) = (1-\alpha)R(\omega_k, t-\Delta t) + \alpha x(t)e^{-i\omega_k t}$ [19]—at the candidate fundamental and its expected partials to verify genuine string excitation, suppress subharmonic octave errors, and enforce a 2-hop confirmation before emitting noteOn[1]. While MPM's slightly sharp cent reading is sufficient for discrete MIDI note naming, applications requiring tuner-grade cent precision should measure partial $h = 1$ directly via the Goertzel stage[1].

The fourth stage manages polyphonic chord detection (PolyEstimator) through two distinct operational modes tailored to application context[1]. In interactive music education and score-following scenarios where the application knows which note or chord the user should play next, **Score-Informed Expected-Notes Mode** (setExpectedNotes([midi...])) restricts Stages 2 and 3 to evaluating targeted Goertzel and comb resonators specifically at the expected fundamentals and their partials[1]. When expecting a $C\ \text{major}$ triad $\{C_4, E_4, G_4\}$ ([60, 64, 67]), striking E_4 alone triggers only E_4, playing adjacent semitones ($D_4^{\#}$, F_4) or unrelated pitches (A_3, C_5) triggers nothing, and striking the full triad resolves all three notes in 23–33 ms[1]. Conversely, when unguided free-form chord detection is required, **Blind Polyphonic Mode** (setPolyphony(true)) activates an iterative harmonic summation and cancellation scheme derived from Klapuri's multiple-F_0 estimator[1]. The input spectrum $X(k)$ is first whitened across subbands using a compression exponent $\nu = 0.33$ to obtain $Y(k) = \gamma(k)X(k)$ [29]. The salience $s(\tau)$ of each candidate period τ is then computed by summing weighted harmonic partials[29]:

$$s(\tau) = \sum_{m=1}^{M} g(\tau, m) \max_{k \in \kappa_{\tau,m}} |Y_R(k)|, \quad \text{with} \quad g(\tau, m) = \frac{f_s/\tau + \alpha}{m f_s/\tau + \beta}$$

During each iteration, the dominant period $\hat{\tau} = \arg\max_\tau s(\tau)$ is selected, its harmonic partial magnitudes $Y_D(k)$ are estimated around bins $\langle mK/\hat{\tau} \rangle$, and those partials are subtracted from the residual spectrum via $Y_R(k) \leftarrow \max(0, Y_R(k) - d \cdot Y_D(k))$ before searching for the next concurrent note[29]. While this blind cancellation stage accurately resolves mid- and high-register triads (C_4–E_4–G_4, C_5–E_5–G_5) and four-note seventh chords ($A\ \text{minor 7}$) in 60–81 ms at 45 µs mean CPU per hop, it struggles on open fifths ($\{C_4, G_4\}$ yields 1/2 plus ghost notes), semitone clusters, and low-register voicings ($\{G_2, D_3, B_3, D_4\}$ yields 0/4), and adds ghost notes on stiff upright bass strings[1]. Consequently, setPolyphony(false) should remain the default configuration unless blind mid-register chord recognition is explicitly needed[1].

Engine Operational Mode	API Configuration	Target Use Case	Latency Profile	Accuracy & Behavioral Characteristics
Free Monophonic / Melodic Cascade	setExpectedNotes([]), setPolyphony(false)	Solo melodies, scales, sight-reading, tuner	4–6.7 ms treble, 12–18 ms mid, 44 ms bass	88/88 keys (100%) on grand; 24/27 bass on 4× stiff upright; 30 µs/hop[1]
Score-Informed Expected Mode	setExpectedNotes([60, 64, 67])	Interactive lessons, guided chord/melody practice	23–33 ms for full 3-note triads	Rejects adjacent semitones ($D_4^\#$, F_4); octave-above ambiguity if expecting p and $p+12$ [cite: 1]
Blind Polyphonic Chord Mode	setExpectedNotes([]), setPolyphony(true)	Free-form mid/high triad and 7th-chord detection	60–81 ms for 3- and 4-note chords	Resolves C maj, C_5 maj, $Am7$ cleanly; unreliable on fifths and low bass chords; 45 µs/hop[1]

Production Engine v1.0 Specification (Currently Implemented Baseline)

The currently implemented **v1.0 piano-engine baseline** is architected for React Native's New Architecture (React Native 0.82+ through 0.87) using Pure C++ TurboModules[1]. The subsections below document the exact v1.0 files and code currently in place[1].

v1.0 Project File Hierarchy and Module Mapping

Source Path	Target Application Path	Architectural Responsibility (v1.0 Baseline)
shared/SampleRing.h	<app>/shared/SampleRing.h	Cache-aligned (alignas(64)) lock-free SPSC ring buffer for real-time PCM transport[1]
shared/Cascade.h, Mpm.h, Goertzel.h, PolyEstimator.h, Fft.h	<app>/shared/	Portable C++17 DSP core implementing the 4-stage gated cascade, MPM autocorrelation, Goertzel resonators, and Klapuri polyphony[1]
shared/Engine.h, AudioInput.h, NativePianoEngine.{h,cpp}	<app>/shared/	Analysis worker thread, hardware abstraction interface, and JSI C++ TurboModule binding[1]
specs/NativePianoEngine.ts	<app>/specs/NativePianoEngine.ts	React Native Codegen TypeScript specification (filename must begin with Native)[1]
src/pianoEngine.ts, src/ExampleScreen.tsx	<app>/src/	High-level TypeScript wrapper, usePianoDetector() React hook, and diagnostic UI[1]
android/app/src/main/jni/OboeInput.cpp, CMakeLists.txt	<app>/android/app/src/main/jni/	Android low-latency Oboe microphone capture and NDK CMake build script[1]
ios/IosAudioInput.mm, NativePianoEngineProvider.{h,mm}	<app>/ios/<AppName>/	iOS RemoteIO AudioUnit microphone capture and Objective-C++ TurboModule provider[1]

test/run.sh	<app>/test/run.sh	Standalone desktop C++17 benchmark (./run.sh quick|full|upright) for accuracy, CPU, and threaded pacing[1]

v1.0 Step 1: Lock-Free Cache-Aligned SPSC SampleRing (shared/SampleRing.h)

This v1.0 header implements the wait-free ring buffer connecting the OS audio callback to the analysis thread, incorporating 64-byte cache line alignment to prevent MESI false sharing and power-of-two bitmasking for branchless wrap-around[1]:

C++

```
// shared/SampleRing.h (v1.0 Implemented Baseline)
#pragma once
#include <atomic>
#include <cstddef>
#include <cstdint>
#include <cstring>
#include <algorithm>

namespace piano {

template <std::size_t CapacityPow2 = 16384>
class SampleRing {
  static_assert((CapacityPow2 & (CapacityPow2 - 1)) == 0,
                "CapacityPow2 must be a power of two");
  static constexpr std::size_t kMask = CapacityPow2 - 1;

public:
  SampleRing() = default;

  // Called exclusively by the OS real-time audio callback (Producer)
  std::size_t push(const float* data, std::size_t count) noexcept {
    const std::size_t tail = tail_.load(std::memory_order_relaxed);
    const std::size_t head = head_.load(std::memory_order_acquire);
    const std::size_t avail = CapacityPow2 - (tail - head);
    const std::size_t toWrite = std::min(count, avail);
```

```
  for (std::size_t i = 0; i < toWrite; ++i) {
    buffer_[(tail + i) & kMask] = data[i];
  }
  tail_.store(tail + toWrite, std::memory_order_release);
  return toWrite;
 }

 // Available samples ready for reading by the analysis thread
 std::size_t available() const noexcept {
  const std::size_t tail = tail_.load(std::memory_order_acquire);
  const std::size_t head = head_.load(std::memory_order_relaxed);
  return tail - head;
 }

 // Peek into the latest window of size W
 void peekWindow(float* dst, std::size_t windowSize) const noexcept {
  const std::size_t head = head_.load(std::memory_order_relaxed);
  const std::size_t start = head - windowSize;
  for (std::size_t i = 0; i < windowSize; ++i) {
    dst[i] = buffer_[(start + i) & kMask];
  }
 }

 // Advance read cursor by one hop (256 samples) in the Consumer thread
 void consume(std::size_t hopSize) noexcept {
  const std::size_t head = head_.load(std::memory_order_relaxed);
  head_.store(head + hopSize, std::memory_order_release);
 }

private:
 alignas(64) std::atomic<std::size_t> head_{4096};
 alignas(64) std::atomic<std::size_t> tail_{4096};
 alignas(64) float buffer_[CapacityPow2]{};
};

} // namespace piano
```

v1.0 Step 2: Native Microphone Capture (OboeInput.cpp & IosAudioInput.mm)

On Android, OboeInput.cpp configures an exclusive low-latency input stream at 48 kHz and pushes incoming frames directly into SampleRing without blocking[1]:

C++
```
// android/app/src/main/jni/OboeInput.cpp (v1.0 Implemented Baseline)
#include "AudioInput.h"
#include "SampleRing.h"
#include <oboe/Oboe.h>

namespace piano {

class OboeAudioInput final : public AudioInput, public oboe::AudioStreamDataCallback {
public:
 explicit OboeAudioInput(SampleRing& ring) : ring_(ring) {}

 bool start(int sampleRate = 48000, int framesPerHop = 256) override {
  oboe::AudioStreamBuilder builder;
  builder.setDirection(oboe::Direction::Input)
       ->setPerformanceMode(oboe::PerformanceMode::LowLatency)
       ->setSharingMode(oboe::SharingMode::Exclusive)
       ->setFormat(oboe::AudioFormat::Float)
       ->setChannelCount(oboe::ChannelCount::Mono)
       ->setSampleRate(sampleRate)
       ->setFramesPerDataCallback(framesPerHop)
       ->setInputPreset(oboe::InputPreset::Unprocessed)
       ->setDataCallback(this);

  if (builder.openStream(stream_) != oboe::Result::OK) return false;
  return stream_->requestStart() == oboe::Result::OK;
 }

 void stop() override {
  if (stream_) {
   stream_->stop();
   stream_->close();
   stream_.reset();
  }
 }

 oboe::DataCallbackResult onAudioReady(
    oboe::AudioStream* /*stream*/, void* audioData, int32_t numFrames) override {
  ring_.push(static_cast<const float*>(audioData), static_cast<std::size_t>(numFrames));
  return oboe::DataCallbackResult::Continue;
```

```
 }

private:
  SampleRing& ring_;
  std::shared_ptr<oboe::AudioStream> stream_;
};

} // namespace piano
```

On iOS (ios/IosAudioInput.mm), the AVAudioSession is initialized with AVAudioSessionCategoryPlayAndRecord and AVAudioSessionModeMeasurement at a preferred sample rate of 48000 Hz and an I/O buffer duration of $256/48000 \approx 0.00533\ \text{s}$, driving a RemoteIO (kAudioUnitSubType_RemoteIO) input render callback that invokes ring_.push(...)[1].

v1.0 Step 3: Baseline Gated-Cascade Configuration and Goertzel Resonator (shared/Cascade.h)

In v1.0, Cascade orchestrates the four-stage pipeline on every 256-sample hop using static default thresholds in Cascade::Config[1]:

C++

```
// shared/Cascade.h (v1.0 Implemented Baseline)
#pragma once
#include <cmath>
#include <cstdint>
#include <vector>

namespace piano {

struct NoteEvent {
  enum Type : uint8_t { NoteOn = 1, NoteOff = 0 } type;
  int midi;
  float cents;
  float velocity;
  double tMs;
};

struct Config {
  int sampleRate = 48000;
  int hopSize = 256;              // 5.333 ms per hop @ 48 kHz
```

```
  float gateRmsDb = -48.0f;          // Rejects ambient noise (-60/-40/-30 dBFS)
  float onsetFluxRatio = 1.85f;      // Spectral/energy rise ratio for attack
  float mpmClarityShort = 0.88f;     // Threshold to accept 1024-sample MPM
  float mpmClarityLong = 0.80f;      // Threshold for escalated 4096-sample MPM
  float goertzelVerifyRatio = 0.12f;  // Min partial energy relative to total frame
  int confirmHops = 2;               // 2-hop debounce for zero false positives
  bool polyphonyEnabled = false;     // Stage 4 experimental chord estimator
};

class Goertzel {
public:
  // Single-bin DFT evaluation at target frequency fHz over N samples
  static float magnitudeSquared(const float* x, int N, float fHz, int sr) noexcept {
    const float omega = 2.0f * static_cast<float>(M_PI) * fHz / static_cast<float>(sr);
    const float coeff = 2.0f * std::cos(omega);
    float s0 = 0.0f, s1 = 0.0f, s2 = 0.0f;
    for (int n = 0; n < N; ++n) {
      s0 = x[n] + coeff * s1 - s2;
      s2 = s1;
      s1 = s0;
    }
    return s1 * s1 + s2 * s2 - coeff * s1 * s2;
  }
};

} // namespace piano
```

v1.0 Step 4: React Native Codegen Specification & TypeScript API

In v1.0, specs/NativePianoEngine.ts defines the TurboModule interface and src/pianoEngine.ts exposes the usePianoDetector hook[1]:

TypeScript

```
// specs/NativePianoEngine.ts (v1.0 Implemented Baseline)
import type {TurboModule} from 'react-native';
import {TurboModuleRegistry} from 'react-native';

export type PianoNoteEvent = {
  type: 'noteOn' | 'noteOff';
  midi: number;
  cents: number;
```

```
  velocity: number;
  tMs: number;
};

export type EngineStats = {
  avgUs: number;
  maxUs: number;
  lateHops: number;
};

export interface Spec extends TurboModule {
  startDetector(): boolean;
  stopDetector(): void;
  setExpectedNotes(midiNotes: ReadonlyArray<number>): void;
  setPolyphony(enabled: boolean): void;
  getEngineStats(): EngineStats;
  onPianoEvent(callback: (event: PianoNoteEvent) => void): void;
}

export default TurboModuleRegistry.getEnforcing<Spec>('NativePianoEngine');
```

TypeScript

```
// src/pianoEngine.ts (v1.0 Implemented Baseline)
import {useEffect, useState, useCallback} from 'react';
import NativePianoEngine, {PianoNoteEvent, EngineStats} from '../specs/NativePianoEngine';

export const startDetector = () => NativePianoEngine.startDetector();
export const stopDetector = () => NativePianoEngine.stopDetector();
export const setExpectedNotes = (midi: number[]) =>
NativePianoEngine.setExpectedNotes(midi);
export const setPolyphony = (enabled: boolean) => NativePianoEngine.setPolyphony(enabled);
export const getEngineStats = (): EngineStats => NativePianoEngine.getEngineStats();
export const onPianoEvent = (cb: (ev: PianoNoteEvent) => void) =>
  NativePianoEngine.onPianoEvent(cb);

export function usePianoDetector(options?: {expected?: number[]; polyphony?: boolean}) {
  const [running, setRunning] = useState(false);
  const [active, setActive] = useState<Map<number, PianoNoteEvent>>(new Map());

  useEffect(() => {
```

```
  setExpectedNotes(options?.expected ?? []);
 }, [options?.expected]);

 useEffect(() => {
  setPolyphony(options?.polyphony ?? false);
 }, [options?.polyphony]);

 useEffect(() => {
  onPianoEvent((ev: PianoNoteEvent) => {
   setActive(prev => {
    const next = new Map(prev);
    if (ev.type === 'noteOn') {
     next.set(ev.midi, ev);
    } else {
     next.delete(ev.midi);
    }
    return next;
   });
  });
  return () => {
   stopDetector();
  };
 }, []);

 const start = useCallback(() => {
  if (startDetector()) setRunning(true);
 }, []);

 const stop = useCallback(() => {
  stopDetector();
  setRunning(false);
  setActive(new Map());
 }, []);

 return {running, active, start, stop, getStats: getEngineStats};
}
```

v1.0 Step 5: Build Plumbing for React Native New Architecture (0.82+ / 0.87)

Integrating the C++17 core into a React Native application follows the official Cross-Platform Native Modules (C++) architecture[1]. First, extend package.json with a codegenConfig block

that maps the specs directory to the NativePianoEngineProvider on iOS and the com.yourapp.specs package on Android[1]:

```
JSON
{
  "codegenConfig": {
    "name": "AppSpecs",
    "type": "modules",
    "jsSrcsDir": "specs",
    "android": {
      "javaPackageName": "com.yourapp.specs"
    },
    "ios": {
      "modulesProvider": {
        "NativePianoEngine": "NativePianoEngineProvider"
      }
    }
  }
}
```

On Android, configure android/app/build.gradle to enable Prefab so CMake can link Google's Oboe library, point externalNativeBuild to src/main/jni/CMakeLists.txt, and declare <uses-permission android:name="android.permission.RECORD_AUDIO" /> inside AndroidManifest.xml[1]:

```
Groovy
android {
  buildFeatures {
    prefab true
  }
  externalNativeBuild {
    cmake {
      path "src/main/jni/CMakeLists.txt"
    }
  }
}
dependencies {
  implementation "com.google.oboe:oboe:1.9.0"
}
```

Still on Android, register NativePianoEngine inside android/app/src/main/jni/OnLoad.cpp so the JSI runtime can instantiate the C++ TurboModule directly[1]:

C++
```
#include <NativePianoEngine.h>

std::shared_ptr<facebook::react::TurboModule> cxxModuleProvider(
   const std::string& name,
   const std::shared_ptr<facebook::react::CallInvoker>& jsInvoker) {
 if (name == facebook::react::NativePianoEngine::kModuleName) {
   return std::make_shared<facebook::react::NativePianoEngine>(jsInvoker);
 }
 return nullptr;
}
```

On iOS, add the shared/ directory and ios/* files (IosAudioInput.mm and NativePianoEngineProvider.{h,mm}) directly to the Xcode project, insert the NSMicrophoneUsageDescription permission key into Info.plist, and run cd ios && pod install to generate the JSI bindings[1].

Engine v1.1 Specification: Calibrated Acoustic Fingerprint Gate, Out-of-Range Noise Rejection & Edge-Case Hardening

While **v1.0** provides the native C++ TurboModule plumbing and achieves 88/88 single-note accuracy in synthetic desktop tests[1], live acoustic deployment requires more than just rejecting a whistle: **once the user's piano is calibrated, the engine already knows the exact acoustic boundaries and timbral signature of that specific piano, and must reject every sound that falls outside the calibrated piano profile**[1]. Whether the interfering sound is a human whistle, talking/singing, clapping, coughing, door slams, table knocks, background television, or another musical instrument, **Engine v1.1** uses a **Five-Layer Calibrated Acoustic Fingerprint Gate** alongside split-scale calibration and comb-notch trill tracking to block all non-piano and out-of-calibration sounds.

Summary of v1.0 vs. v1.1 Architectural Upgrades

Subsystem / File	v1.0 (Current	v1.1 (Calibrated Acoustic Fingerprint & Real-World

	Implemented Baseline)	DSP Upgrade)
shared/SampleRing.h	Fixed head - windowSize lookback; no automatic recovery if OS stalls consumer thread[1].	**v1.1 Zero-Drift Ring:** Anchors peekWindow to head + hopSize - windowSize and adds dropStaleBacklog() to prevent cumulative latency drift[12].
shared/Calibration.h	*Not present in v1.0* (Static Cascade::Config constants tuned only on synthetic grand piano)[1].	**v1.1 New File:** Split-Scale Two-Strike Calibrator (C1–C6 + A4 Fast Mode or 16-Key Chromatic Mode) that learns the piano's frequency grid, timbre template partialWeights, in-comb HNR ratio, and dynamic range[27].
shared/Cascade.h	Broadband RMS onset, MPM + basic Goertzel sum; accepts non-piano sounds and locks up on C-D-C-D trills[1].	**v1.1 Hardened Cascade:** Enforces the **5-Layer Calibrated Acoustic Fingerprint Gate** (rejecting all sounds outside the calibrated frequency, timbre, comb-HNR, and dynamic range), plus 1-tap comb notch trill cancellation[7].
src/CalibrationScreen.tsx	*Not present in v1.0* (ExampleScreen.tsx demo only)[1].	**v1.1 New File:** Guided 2-strike UI with live dBFS range, cent offset, piano type classifier, and out-of-calibration noise/wrong-key warning banners[36].

How v1.1 Blocks Everything Outside the Calibration Range (The 5-Layer Acoustic Fingerprint Gate)

Because the two-strike calibration step measures the user's actual piano through their phone's actual microphone in their actual room, PianoProfile captures a complete **Acoustic Fingerprint** of the instrument[1]. During live detection (Cascade::processHop), any incoming audio frame is tested against five strict calibration boundaries; if a sound falls outside **any** boundary, it is immediately discarded as non-piano noise:

1. **Layer 1: Calibrated Key-Range & Exact Cent-Grid Lock ([minCalibratedMidi, maxCalibratedMidi] & keyFreqHz[m] ± maxCentsWindow):** Every key m on the user's piano has a fixed physical string tension and a calibrated fundamental frequency $f_0^{\text{cal}}(m)$ [7]. First, any pitch outside the instrument's calibrated MIDI range [minCalibratedMidi, maxCalibratedMidi] is ignored[8]. Second, because a piano string cannot play quarter-tones or slide between pitches like a human voice, siren, or violin, Cascade::processHop measures the fine cent offset Δc relative to $f_0^{\text{cal}}(m)$ [8]. If $|\Delta c| > \text{calibratedPitchToleranceCents}$ (default ± 22 cents once calibrated), the sound is outside the piano's tuned frequency grid and is rejected[8].
2. **Layer 2: Calibrated In-Comb Harmonic-to-Residual Energy Ratio (Broadband & Percussive Noise Gate):** When a calibrated piano key m is struck, its acoustic power is concentrated inside the narrow Goertzel comb teeth at the stretched partials $f_h(m) = h f_0^{\text{cal}}(m)\sqrt{1 + B(m)h^2}$ for $h = 1 \dots 6$ [7]. Conversely, room noises—such as claps, coughs, footsteps, dish clattering, table knocks, and speech fricatives—distribute energy across the frequencies *between* the harmonic partials[30]. During calibration, the engine measures the piano's **In-Comb Energy Ratio**:

$$\text{CombRatio}(m) = \frac{\sum_{h=1}^{6} E_{\text{Goertzel}}\left(f_h(m)\right)}{E_{\text{totalFrame}}}$$

   and stores the lower bound minCombEnergyRatio. During gameplay, any sound whose in-comb harmonic concentration falls below profile_.minCombEnergyRatio (meaning too much energy lies outside the calibrated harmonic comb teeth) is rejected as broadband room noise[35].
3. **Layer 3: Calibrated Timbre Template Cosine Similarity (partialWeights Matching):** Different sound sources have distinct harmonic amplitude envelopes $\mathbf{w} = [w_1, w_2, \dots, w_6]$ [8]. During calibration, PianoCalibrator records the user's normalized piano timbre vector $\mathbf{w}^{\text{cal}}$ and the valid overtone ratio window [minOvertoneRatio, maxOvertoneRatio][8]. During live detection, the engine extracts the candidate's normalized partial magnitude vector $\mathbf{a} = [\sqrt{E_1}, \sqrt{E_2}, \dots, \sqrt{E_6}]$ and computes the **Cosine Similarity** against the calibrated piano profile[30]:

$$\text{TimbreSimilarity}(\mathbf{a}, \mathbf{w}^{\text{cal}}) = \frac{\sum_{h=1}^{6} a_h w_h^{\text{cal}}}{\sqrt{\sum_{h=1}^{6} a_h^2} \sqrt{\sum_{h=1}^{6} (w_h^{\text{cal}})^2}}$$

- A **whistle or pure tone** has $a_2 \dots a_6 \approx 0$ [16], failing both the minimum overtone bound (overtoneRatio < profile_.minOvertoneRatio) and the timbre similarity threshold.
- **Human speech, singing vowels, guitars, or TV audio** exhibit formant peaks or odd-harmonic dominance that deviate sharply from the piano's calibrated $\mathbf{w}^{\text{cal}}$ template, failing the TimbreSimilarity < profile_.minTimbreSimilarity (0.76) gate and the maximum overtone bound (overtoneRatio > profile_.maxOvertoneRatio)[30].

4. **Layer 4: Calibrated Dynamic Envelope Window ([minVelocityDb, maxVelocityDb]):** Because the user strikes each calibration key twice—**Strike 1 (Soft)** and **Strike 2 (Firm)**—the engine knows the exact dBFS loudness range of the piano at the phone's microphone position ([minVelocityDb - 5 dB, maxVelocityDb + 8 dB]) plus the ambient noise floor calibratedGateDb[1]. Distant background noises below the soft-strike floor or microphone-handling thumps that clip outside the calibrated acoustic envelope are rejected.
5. **Layer 5: Hammer Transient Slope & Inter-Hop Pitch Stability (|\Delta cents| < 18):** A piano string is excited by a felt hammer impulse and remains at a stable frequency across consecutive 5.33 ms hops[1]. Non-piano sounds that slide in pitch between Hop 1 and Hop 2 ($|\text{cents}_t - \text{cents}_{t-1}| \geq 18 \text{ cents}$) are rejected before the 2-hop confirmation can fire[1].

Interfering Non-Piano Sound	**Why Naive Pitch Detectors Falsely Trigger**	**Which v1.1 Calibrated Fingerprint Layer Rejects It**
Human Whistle / Pure Tone	High MPM periodicity clarity ($\approx$) on fundamental f_1 [16].	**Layer 3 (Overtone & Timbre Gate):** overtoneRatio < minOvertoneRatio and low cosine similarity to partialWeights[30].
Human Speech / Singing / Humming	Vocal cords produce harmonic series in C_2–C_6 range[18].	**Layer 1, 3 & 5:** Fails ± 22 cent calibrated grid lock, fails piano

		TimbreSimilarity due to vocal formants, and fails $\lvert\Delta\text{cents}\rvert <$ stability[8].
Claps, Coughs, Knocks, Door Slams	Loud transient spike crosses broadband RMS onset threshold[30].	**Layer 2 (In-Comb HNR Gate):** Energy is spread across all frequencies; combRatio < minCombEnergyRatio rejects it immediately[30].
Background TV / Other Instruments	Pitched audio within room[8].	**Layer 3 & 4:** Spectral envelope $\mathbf{a}$ mismatches calibrated piano partialWeights template and hammer attack slope[30].
Out-of-Range / Uncalibrated Frequencies	Random high-frequency squeaks or low rumble (< 27.5 Hz or > 4186 Hz)[8].	**Layer 1 (Key-Range & Cent Window):** Restricted strictly to [minCalibratedMidi, maxCalibratedMidi] within ±calibratedPitchToleranceCents[8].

v1.1 Research on Full-Piano Calibration: Why Split-Scale Multi-Octave Sampling (C_1–C_6) Beats Middle-Octave-Only (C_4–B_4)

A critical design question for **v1.1** is whether prompting the user to strike all 12 chromatic keys of the middle octave ($C_4, C_4^\#, D_4, D_4^\#, E_4, F_4, F_4^\#, G_4, G_4^\#, A_4, A_4^\#, B_4$) twice—24 strikes total—is the optimal method to calibrate an entire 88-key piano. Acoustic research and empirical methodologies from industry-standard Electronic Tuning Devices (ETDs) such as **TuneLab Pro** and **PianoMeter** demonstrate that **sampling only the middle octave (C_4–B_4) is sub-optimal for full-piano calibration**, and establish a faster, mathematically superior **Split-Scale Multi-Octave Anchor + Continuous Online Adaptation** technique[27].

1. **Extreme Redundancy Within Octave 4:** Within any single plain-steel octave (C_4–B_4), adjacent strings share nearly identical wire metallurgy, speaking-length scaling ratios, and bridge termination impedance. Most pianos exhibit a uniform inharmonicity progression within a section, meaning that measuring 12 consecutive semitones in

Octave 4 collects 12 redundant data points on the exact same segment of the treble bridge while leaving the remaining 76 keys unmeasured[27].

2. **Capturing the U-Shaped 88-Key Inharmonicity Curve:** Across an 88-key piano (A_0 to C_8), the string inharmonicity constant $B(m)$ is **not constant or linear**[27]. In a well-scaled piano, $B(m)$ reaches its lowest value around Octave 2 (C_2–B_2), increases slightly as you move down into the heavy copper-wound bass strings toward A_0, and increases *substantially* (exponentially) as you move up the plain-steel treble strings toward C_8 [27]. Measuring only C_4–B_4 blinds the curve fitter to both the Octave 2 minimum and the steep high-treble climb[27].
3. **The Bass-to-Treble Bridge Discontinuity ("Split-Scale Jump"):** Acoustic pianos—particularly compact uprights and spinets—divide their strings across two physically separate bridges: the copper-wound bass bridge (A_0–B_2) and the plain-steel treble bridge (C_3–C_8)[32]. Spinets and small uprights exhibit a sharp jump in inharmonicity at the bass-to-treble transition[32]. TuneLab's **Split-Scale Mode** calculates two separate exponential inharmonicity curves—one for each half of the scale—and defaults to sampling the C notes across the octaves ($C_1, C_2, C_3, C_4, C_5, C$)[27].

Calibration Technique (v1.1 Options)	Keys Sampled (2 Strikes Each)	Total Strikes & Time	Bass Bridge Fit (A0–B2A_0\text{–}B_2A0–B2)	Treble Bridge Fit (C3–C8C_3\text{–}C_8C3–C8)	Individual Out-of-Tune Key Capture	Overall Verdict for piano-engine v1.1
1. Middle-Octave Chromatic Only	12 keys: $C_4, C_4^{\#}, D$	24 strikes (~40 sec)	**Poor** (Extrapolates blindly across bridge break)[1]	**Moderate** (Misses Octave 5–6 exponential rise)[27]	Only for MIDI 60–71	**Sub-optimal alone**; wastes 24 strikes on 1 octave while missing bass stiffness[1].

2. TuneLab 6-Octave C Sweep (Fast Split-Scale)	6 keys: C_1, C_2, C	**12 strikes (~18 sec)**	**Excellent** (C_1, C_2 lock copper-wound bass curve)[27]	**Excellent** (C_3–C_6 fit full treble slope & stretch)[27]	Interpolated via Railsback curve; refined live	**Best Fast Calibration**; industry-standard ETD sequence[27].
3. v1.1 Hybrid Split-Scale + Middle Chromatic + Online Auto-Refine	**Fast Mode:** C_1, C_2, C (7 keys) **OR Full Mode:** C_1, C_2, C + Live Auto-Listen	**14 strikes (~20 sec)** in Fast Mode	**Excellent** (Exact 2-bridge split-scale regression)[32]	**Excellent** (Captures A_4 reference, C_3–C_6 slope, and mic EQ)[27]	**All 88 keys** refined continuously during play[43]	**v1.1 Standard**; combines TuneLab's C1–C6 split-scale curve with PianoMeter's live adaptation[27].

By combining **TuneLab's C1–C6 Split-Scale sequence**[27] with the **Two-Strike Dynamic Range protocol**[34] and **PianoMeter's always-listening online refinement**[43], v1.1 models both bridges across the bass/treble break at $m_{\text{break}} = 48$ (C_3):

$$\ln B_{\text{bass}}(m) = a_B m + b_B \quad (m < 48), \qquad \ln B_{\text{treble}}(m) = a_T m + b_T \quad (m \geq 48)$$

From $B(m)$, v1.1 synthesizes the 88-key Railsback octave stretch curve (using the 6:3 partial interval in the bass and 4:2 partial interval in the treble, yielding ≈ -10 cents at A_0 and $+30$ cents at C_8)[37]:

$$f_0(m) = f_{A4} \cdot 2^{(m-69)/12} \cdot \sqrt{\frac{1 + B(m) \cdot h_{\text{tune}}^2}{1 + B(69) \cdot h_{\text{tune}}^2}}$$

Additional Real-World Playing Permutations Solved in v1.1

- **Rapid Trills (C-D-C-D-C-D) via Active-Note Comb Cancellation:** When a pianist alternates rapidly between two adjacent notes such as C_4 (261.6 Hz) and D_4 (293.7

Hz), the undamped sustain tail of C_4 overlaps the attack of D_4 inside the 1024-sample window, causing two-tone wave beating that collapses MPM autocorrelation clarity and saturates broadband RMS onset detection. In v1.1, applying a fast 1-tap time-domain comb notch filter $y[n] = x[n] - \alpha_{\text{cancel}}\, x[n - \tau_{\text{active}}]$ at the currently active note's period τ_{active} strips the ringing note out of the analysis buffer in under 3 µs, allowing the newly struck note (D_4) to lock cleanly while immediately closing C_4 [1].

- **Fast Same-Key Repetition (C-C-C-C):** Tracking the per-note harmonic envelope $E_1(m) + E_2(m)$ detects when an active key has entered decay (dropped by ≥ 5 dB from its peak) and then experiences a fresh hammer re-strike ($\geq +4.5$ dB jump), firing an immediate NoteOff + NoteOn pair on the same MIDI key[1].
- **Calibration Sustain Bleed (WaitReleaseValley):** Because a struck piano string rings for several seconds, a fixed 150 ms timer between calibration strikes allows Strike 1's sustain tail to false-trigger Strike 2. Requiring an envelope valley drop followed by a fresh positive dB/hop attack slope guarantees each calibration strike is an independent hammer blow[36].
- **Wrong-Key Strike During Guided Calibration:** Scanning adjacent semitones ($\pm 1, \pm 2$) during calibration ensures that if the user accidentally hits D_4 when asked for C_4 , the calibrator flags a wrong-key warning (state = -1) instead of corrupting the C_4 frequency slot[36].

v1.1 Upgrade 1: Zero-Drift SampleRing.h (shared/SampleRing.h)

In **v1.1**, update shared/SampleRing.h to anchor peekWindow to head + hopSize - windowSize and add dropStaleBacklog() so OS thread stalls cannot cause permanent latency drift[1]:

C++

```
// shared/SampleRing.h (v1.1 Zero-Drift Upgrade)
#pragma once
#include <atomic>
#include <cstddef>
#include <cstdint>
#include <cstring>
#include <algorithm>

namespace piano {

template <std::size_t CapacityPow2 = 16384>
class SampleRing {
```

```
  static_assert((CapacityPow2 & (CapacityPow2 - 1)) == 0,
                "CapacityPow2 must be a power of two");
  static constexpr std::size_t kMask = CapacityPow2 - 1;

public:
  SampleRing() = default;

  std::size_t push(const float* data, std::size_t count) noexcept {
    const std::size_t tail = tail_.load(std::memory_order_relaxed);
    const std::size_t head = head_.load(std::memory_order_acquire);
    const std::size_t avail = CapacityPow2 - (tail - head);
    const std::size_t toWrite = std::min(count, avail);

    for (std::size_t i = 0; i < toWrite; ++i) {
      buffer_[(tail + i) & kMask] = data[i];
    }
    tail_.store(tail + toWrite, std::memory_order_release);
    return toWrite;
  }

  std::size_t available() const noexcept {
    const std::size_t tail = tail_.load(std::memory_order_acquire);
    const std::size_t head = head_.load(std::memory_order_relaxed);
    return tail - head;
  }

  // v1.1: Fast-forwards read cursor if OS scheduler stalled the consumer thread (>4 hops = >21.3
ms)
  bool dropStaleBacklog(std::size_t hopSize = 256, std::size_t maxBacklogHops = 4) noexcept {
    const std::size_t tail = tail_.load(std::memory_order_acquire);
    const std::size_t head = head_.load(std::memory_order_relaxed);
    if (tail - head > hopSize * maxBacklogHops) {
      head_.store(tail - hopSize, std::memory_order_release);
      return true; // Indicates a lateHop backlog was recovered
    }
    return false;
  }

  // v1.1: Copies `windowSize` samples ending exactly at the end of the current `hopSize` block
  void peekWindow(float* dst, std::size_t windowSize, std::size_t hopSize = 256) const noexcept
{
    const std::size_t endCursor = head_.load(std::memory_order_relaxed) + hopSize;
    const std::size_t startCursor = endCursor - windowSize;
```

```
    for (std::size_t i = 0; i < windowSize; ++i) {
      dst[i] = buffer_[(startCursor + i) & kMask];
    }
  }

  void consume(std::size_t hopSize = 256) noexcept {
    const std::size_t head = head_.load(std::memory_order_relaxed);
    head_.store(head + hopSize, std::memory_order_release);
  }

private:
  alignas(64) std::atomic<std::size_t> head_{4096};
  alignas(64) std::atomic<std::size_t> tail_{4096};
  alignas(64) float buffer_[CapacityPow2]{};
};

} // namespace piano
```

v1.1 Upgrade 2: Hardened Gated-Cascade Core with 5-Layer Calibrated Acoustic Fingerprint Gate (shared/Cascade.h)

In **v1.1**, shared/Cascade.h enforces the **Calibrated Acoustic Fingerprint Gate** (strictCalibrationFilter = true), rejecting any sound whose frequency offset, in-comb harmonic-to-noise ratio (minCombEnergyRatio), overtone ratio ([minOvertoneRatio, maxOvertoneRatio]), timbre cosine similarity (minTimbreSimilarity), or dynamic level ([minVelocityDb, maxVelocityDb]) falls outside the calibrated piano profile, while supporting 1-tap comb notch trill cancellation[1]:

C++

```
// shared/Cascade.h (v1.1 Hardened Upgrade with Full Out-of-Calibration Noise Rejection)
#pragma once
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdint>
#include <vector>

namespace piano {

struct NoteEvent {
```

```
  enum Type : uint8_t { NoteOn = 1, NoteOff = 0 } type;
  int midi;
  float cents;
  float velocity;
  double tMs;
};

struct Config {
  int sampleRate = 48000;
  int hopSize = 256;               // 5.333 ms per hop @ 48 kHz
  float gateRmsDb = -48.0f;         // Rejects ambient noise (-60/-40/-30 dBFS)
  float onsetFluxRatio = 1.85f;      // Spectral/energy rise ratio for attack
  float mpmClarityShort = 0.88f;     // Threshold to accept 1024-sample MPM
  float mpmClarityLong = 0.80f;      // Threshold for escalated 4096-sample MPM
  float goertzelVerifyRatio = 0.12f;  // Min partial energy relative to total frame
  int confirmHops = 2;              // 2-hop debounce for zero false positives
  bool polyphonyEnabled = false;     // Stage 4 experimental chord estimator
  bool strictCalibrationFilter = true;// v1.1: Reject all sounds outside calibrated PianoProfile
};

struct PianoProfile {
  bool isCalibrated = false;
  int minCalibratedMidi = 21;        // Lowest allowed MIDI key (default A0 = 21)
  int maxCalibratedMidi = 108;       // Highest allowed MIDI key (default C8 = 108)
  float a4ReferenceHz = 440.0f;
  float ambientNoiseDb = -56.0f;
  float calibratedGateDb = -48.0f;
  float minVelocityDb = -40.0f;      // Softest calibrated piano strike (dBFS)
  float maxVelocityDb = -4.0f;       // Firmest calibrated piano strike (dBFS)
  float calibratedFluxRatio = 1.85f;
  float calibratedPitchWindowCents = 22.0f; // Max allowed deviation from calibrated
keyFreqHz[m]
  float minCombEnergyRatio = 0.18f;   // Min ratio of (E1..E6) / TotalFrameEnergy (blocks
claps/coughs)
  float minOvertoneRatio = 0.045f;    // Min (E2..E4)/E1 (blocks pure sine whistles)
  float maxOvertoneRatio = 4.50f;     // Max (E2..E4)/E1 (blocks buzzy non-piano noise/speech)
  float minTimbreSimilarity = 0.76f;  // Min cosine similarity to calibrated partialWeights[]
  float bassInharmonicityB0 = 0.00018f;
  float bassSlope = -0.035f;
  float trebleInharmonicityB0 = 0.00004f;
  float trebleSlope = 0.052f;
  float meanInharmonicityB = 0.00035f; // ~0.0 digital, ~0.0003 grand, >=0.0007 upright
  std::array<float, 128> keyFreqHz{};
```

```
  std::array<float, 128> inharmonicityB{};
  std::array<float, 8> partialWeights{1.0f, 0.65f, 0.42f, 0.28f, 0.18f, 0.12f, 0.08f, 0.05f};
};

class Goertzel {
public:
  static float magnitudeSquared(const float* x, int N, float fHz, int sr) noexcept {
    const float omega = 2.0f * static_cast<float>(M_PI) * fHz / static_cast<float>(sr);
    const float coeff = 2.0f * std::cos(omega);
    float s0 = 0.0f, s1 = 0.0f, s2 = 0.0f;
    for (int n = 0; n < N; ++n) {
      s0 = x[n] + coeff * s1 - s2;
      s2 = s1;
      s1 = s0;
    }
    return s1 * s1 + s2 * s2 - coeff * s1 * s2;
  }
};

class Cascade {
public:
  Cascade() noexcept {
    for (int m = 0; m < 128; ++m) {
      profile_.keyFreqHz[m] = 440.0f * std::pow(2.0f, (m - 69) / 12.0f);
      profile_.inharmonicityB[m] = 0.00035f;
    }
  }

  void setConfig(const Config& cfg) noexcept { config_ = cfg; }
  void setProfile(const PianoProfile& prof) noexcept { profile_ = prof; }
  const PianoProfile& profile() const noexcept { return profile_; }

  void setExpectedNotes(const std::vector<int>& expected) noexcept {
    expectedNotes_ = expected;
  }

  // Runs every 256-sample hop (5.33 ms @ 48 kHz)
  void processHop(const float* window4096, const float* hop256, double currentTimeMs,
                  std::vector<NoteEvent>& outEvents) noexcept {
    // Stage 1: Compute hop RMS and short-term attack derivative
    float sumSq = 0.0f;
    for (int i = 0; i < 256; ++i) sumSq += hop256[i] * hop256[i];
    const float rmsLinear = std::sqrt(sumSq / 256.0f + 1e-12f);
```

```
  const float rmsDb = 20.0f * std::log10(rmsLinear);
  const float deltaDb = rmsDb - prevHopDb_;
  prevHopDb_ = rmsDb;

  const float gateDb = profile_.isCalibrated ? profile_.calibratedGateDb : config_.gateRmsDb;

  // Layer 4A: Global silence / below-calibrated-range check -> exit in ~0.4 us
  if (rmsDb < gateDb) {
   if (activeMidi_ >= 0 && ++releaseHops_ >= 3) {
    outEvents.push_back({NoteEvent::NoteOff, activeMidi_, 0.0f, 0.0f, currentTimeMs});
    activeMidi_ = -1;
   }
   confirmCount_ = 0;
   return;
  }

  // Layer 4B: If calibrated, reject extreme overload bangs/thumps above calibrated max + 8 dB
  if (profile_.isCalibrated && config_.strictCalibrationFilter) {
   if (rmsDb < profile_.minVelocityDb - 6.0f || rmsDb > profile_.maxVelocityDb + 8.0f) {
    confirmCount_ = 0;
    return;
   }
  }
  releaseHops_ = 0;

  // Stage 2A: Per-note energy tracking & Active-Note Comb Cancellation for fast C-D-C-D
trills
  const float* analysisBuf1024 = window4096 + (4096 - 1024);
  float combFiltered1024[1024];

  if (activeMidi_ >= 21 && activeMidi_ <= 108) {
   const float activeF0 = getKeyFreq(activeMidi_);
   const float B = profile_.inharmonicityB[activeMidi_];
   const float e1 = Goertzel::magnitudeSquared(analysisBuf1024, 1024, activeF0,
config_.sampleRate);
   const float f2 = 2.0f * activeF0 * std::sqrt(1.0f + 4.0f * B);
   const float e2 = Goertzel::magnitudeSquared(analysisBuf1024, 1024, f2,
config_.sampleRate);
   const float activeNoteDb = 10.0f * std::log10((e1 + e2) / 1024.0f + 1e-12f);

   // Same-key rapid repetition (C-C-C-C): Note decayed >= 5 dB then jumped >= +4.5 dB
   if (activeNoteHasDecayed_ && (activeNoteDb - prevActiveNoteDb_ > 4.5f) && deltaDb >
3.0f) {
```

```
      outEvents.push_back({NoteEvent::NoteOff, activeMidi_, 0.0f, 0.0f, currentTimeMs});
      outEvents.push_back({NoteEvent::NoteOn, activeMidi_, lastCents_,
                           dbToVelocity(rmsDb), currentTimeMs});
      activePeakDb_ = activeNoteDb;
      activeNoteHasDecayed_ = false;
      prevActiveNoteDb_ = activeNoteDb;
      return;
    }
    if (activeNoteDb < activePeakDb_ - 5.0f) {
      activeNoteHasDecayed_ = true;
    }
    activePeakDb_ = std::max(activePeakDb_, activeNoteDb);
    prevActiveNoteDb_ = activeNoteDb;

    // Apply 1-tap comb notch filter to cancel the currently ringing note (e.g., C4)
    // so that if the user strikes D4 in a fast C-D-C-D trill, D4 stands out cleanly
    const int periodSamples = std::clamp(
        static_cast<int>(std::lround(config_.sampleRate / activeF0)), 8, 1023);
    for (int i = 0; i < 1024; ++i) {
      const float delayed = window4096[4096 - 1024 + i - periodSamples];
      combFiltered1024[i] = analysisBuf1024[i] - 0.82f * delayed;
    }
    analysisBuf1024 = combFiltered1024;
  }

  // Stage 2B & Stage 3: Search Within Calibrated Range & Apply 5-Layer Acoustic Fingerprint
Gate
  int bestMidi = -1;
  float bestSalience = 0.0f;
  float bestCombEnergy = 0.0f;

  const bool hasExpected = !expectedNotes_.empty();
  // Layer 1A: Restrict search strictly to calibrated MIDI key range
  const int startMidi = profile_.isCalibrated ? profile_.minCalibratedMidi : 21;
  const int endMidi = profile_.isCalibrated ? profile_.maxCalibratedMidi : 108;

  for (int m = startMidi; m <= endMidi; ++m) {
    if (m == activeMidi_) continue;
    if (hasExpected && std::find(expectedNotes_.begin(), expectedNotes_.end(), m) ==
expectedNotes_.end()) {
      continue;
    }
```

```
const float f0 = getKeyFreq(m);
const float B = profile_.inharmonicityB[m];
const int N = (m < 48) ? 4096 : 1024;
const float* buf = (m < 48) ? window4096 : analysisBuf1024;

// Evaluate first 6 stretched partials of the calibrated piano key
std::array<float, 6> ePartials{};
std::array<float, 6> aPartials{};
float combEnergySum = 0.0f;
for (int h = 1; h <= 6; ++h) {
  const float fh = h * f0 * std::sqrt(1.0f + B * static_cast<float>(h * h));
  if (fh >= config_.sampleRate * 0.46f) break;
  const float eh = Goertzel::magnitudeSquared(buf, N, fh, config_.sampleRate);
  ePartials[h - 1] = eh;
  aPartials[h - 1] = std::sqrt(std::max(0.0f, eh));
  combEnergySum += eh;
}

const float e1 = ePartials[0];
const float overtoneSum = ePartials[1] + ePartials[2] + ePartials[3];

// Layer 3A: Calibrated Overtone Ratio Gate (Rejects whistles AND buzzy non-piano sounds)
if (m < 96 && e1 > 1e-9f) {
  const float overtoneRatio = overtoneSum / e1;
  const float minRatio = (m < 48) ? 0.35f : profile_.minOvertoneRatio;
  const float maxRatio = (m < 48) ? 12.0f : profile_.maxOvertoneRatio;
  if (overtoneRatio < minRatio || overtoneRatio > maxRatio) continue;
}

// Layer 3B: Calibrated Timbre Template Cosine Similarity Gate
// Compares live partial amplitudes aPartials[0..5] against calibrated partialWeights[0..5]
if (profile_.isCalibrated && config_.strictCalibrationFilter && m >= 36 && m <= 88) {
  float dot = 0.0f, normA = 0.0f, normW = 0.0f;
  for (int h = 0; h < 6; ++h) {
    dot += aPartials[h] * profile_.partialWeights[h];
    normA += aPartials[h] * aPartials[h];
    normW += profile_.partialWeights[h] * profile_.partialWeights[h];
  }
  const float cosineSim = dot / (std::sqrt(normA * normW) + 1e-9f);
  if (cosineSim < profile_.minTimbreSimilarity) {
    continue; // Rejects voice, guitar, TV, and non-piano timbres!
  }
}
```

```
    // Missing-fundamental-safe salience using calibrated partial envelope weights
    const float salience = ePartials[0] * profile_.partialWeights[0] +
                           ePartials[1] * profile_.partialWeights[1] +
                           ePartials[2] * profile_.partialWeights[2] +
                           ePartials[3] * profile_.partialWeights[3];
    if (salience > bestSalience) {
      bestSalience = salience;
      bestCombEnergy = combEnergySum;
      bestMidi = m;
    }
  }

  if (bestMidi < 0) {
    confirmCount_ = 0;
    return;
  }

  // Layer 2: In-Comb Harmonic-to-Residual Energy Ratio (Rejects claps, knocks, coughs,
footsteps)
  const int bestN = (bestMidi < 48) ? 4096 : 1024;
  const float* bestBuf = (bestMidi < 48) ? window4096 : analysisBuf1024;
  float windowEnergy = 0.0f;
  for (int i = 0; i < bestN; ++i) windowEnergy += bestBuf[i] * bestBuf[i];
  // Normalize Goertzel energy (scales as N/2) against time-domain window energy
  const float normalizedCombRatio =
      (bestCombEnergy / (0.5f * static_cast<float>(bestN))) / (windowEnergy + 1e-9f);
  const float minCombReq =
      (profile_.isCalibrated && config_.strictCalibrationFilter)
          ? profile_.minCombEnergyRatio
          : config_.goertzelVerifyRatio;

  if (normalizedCombRatio < minCombReq) {
    confirmCount_ = 0; // Rejected: broadband or percussive room noise outside harmonic
comb
    return;
  }

  // Layer 1B: Calibrated Cent-Grid Lock (Rejects off-grid frequencies between piano keys)
  const float bestCents = estimateFineCents(bestBuf, bestN, getKeyFreq(bestMidi));
  if (profile_.isCalibrated && config_.strictCalibrationFilter) {
    if (std::fabs(bestCents) > profile_.calibratedPitchWindowCents) {
      confirmCount_ = 0; // Rejected: pitch lies outside the calibrated piano tuning grid
```

```
      return;
    }
  }

  // Layer 5: 2-Hop Confirmation + Pitch-Scoop Gate (Rejects vocal scoops & frequency slides)
  if (bestMidi == candidateMidi_ && std::fabs(bestCents - candidateCents_) < 18.0f) {
    if (++confirmCount_ >= config_.confirmHops) {
      if (activeMidi_ >= 0 && activeMidi_ != bestMidi) {
        outEvents.push_back({NoteEvent::NoteOff, activeMidi_, 0.0f, 0.0f, currentTimeMs});
      }
      activeMidi_ = bestMidi;
      lastCents_ = bestCents;
      activePeakDb_ = rmsDb;
      prevActiveNoteDb_ = rmsDb;
      activeNoteHasDecayed_ = false;
      confirmCount_ = 0;

      // Continuously refine this individual key's fundamental frequency if within +/- 20 cents
      if (std::fabs(bestCents) < 20.0f) {
        const float measuredHz = getKeyFreq(bestMidi) * std::pow(2.0f, bestCents / 1200.0f);
        profile_.keyFreqHz[bestMidi] = 0.90f * profile_.keyFreqHz[bestMidi] + 0.10f * measuredHz;
      }

      outEvents.push_back({NoteEvent::NoteOn, activeMidi_, bestCents,
                           dbToVelocity(rmsDb), currentTimeMs});
    }
  } else {
    candidateMidi_ = bestMidi;
    candidateCents_ = bestCents;
    confirmCount_ = 1;
  }
}

private:
  float getKeyFreq(int midi) const noexcept {
    if (midi >= 0 && midi < 128 && profile_.keyFreqHz[midi] > 20.0f) {
      return profile_.keyFreqHz[midi];
    }
    return 440.0f * std::pow(2.0f, (midi - 69) / 12.0f);
  }

  float estimateFineCents(const float* buf, int N, float centerHz) const noexcept {
    const float magCenter = Goertzel::magnitudeSquared(buf, N, centerHz, config_.sampleRate);
```

```
  const float hzMinus = centerHz * std::pow(2.0f, -15.0f / 1200.0f);
  const float hzPlus = centerHz * std::pow(2.0f, +15.0f / 1200.0f);
  const float magMinus = Goertzel::magnitudeSquared(buf, N, hzMinus, config_.sampleRate);
  const float magPlus = Goertzel::magnitudeSquared(buf, N, hzPlus, config_.sampleRate);
  const float denom = magMinus - 2.0f * magCenter + magPlus;
  if (std::fabs(denom) < 1e-9f) return 0.0f;
  const float delta = 0.5f * (magMinus - magPlus) / denom;
  return std::clamp(delta * 15.0f, -35.0f, 35.0f);
 }

 float dbToVelocity(float db) const noexcept {
  const float lo = profile_.isCalibrated ? profile_.minVelocityDb : -42.0f;
  const float hi = profile_.isCalibrated ? profile_.maxVelocityDb : -6.0f;
  return std::clamp((db - lo) / std::max(6.0f, hi - lo), 0.05f, 1.0f);
 }

 Config config_{};
 PianoProfile profile_{};
 std::vector<int> expectedNotes_{};
 int activeMidi_ = -1;
 int candidateMidi_ = -1;
 int confirmCount_ = 0;
 int releaseHops_ = 0;
 float candidateCents_ = 0.0f;
 float lastCents_ = 0.0f;
 float prevHopDb_ = -60.0f;
 float activePeakDb_ = -60.0f;
 float prevActiveNoteDb_ = -60.0f;
 bool activeNoteHasDecayed_ = false;
};

} // namespace piano
```

v1.1 Upgrade 3: C++ Split-Scale & Acoustic Fingerprint Calibration Engine (shared/Calibration.h)

In **v1.1**, shared/Calibration.h captures the two strikes per key, rejects any non-piano sound or wrong key during calibration, and computes the exact **Acoustic Fingerprint** bounds (minCombEnergyRatio, minOvertoneRatio, maxOvertoneRatio, minTimbreSimilarity, calibratedPitchWindowCents, minVelocityDb, maxVelocityDb, and the 88-key keyFreqHz / inharmonicityB tables) so Cascade.h can block all sounds outside the calibration profile[30]:

C++
```
// shared/Calibration.h (New in v1.1 — Captures Full Piano Acoustic Fingerprint)
#pragma once
#include "Cascade.h"
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdint>

namespace piano {

struct CalibrationProgressEvent {
  int currentMidi;          // Target MIDI key (e.g. 24=C1, 36=C2, 48=C3, 60=C4...)
  int keyIndex;             // 0 .. totalKeys - 1
  int totalKeys;            // 7 (Fast Split-Scale) or 16 (Full Chromatic + Octaves)
  int strikeNumber;         // 1 (Soft/Normal) or 2 (Firm)
  int state;                // -2=NonPianoNoiseRejected, -1=WrongKey, 0=NoiseFloor,
                            //  1=WaitStrike1, 2=WaitStrike2, 3=KeyDone, 4=AllDone
  float measuredFreqHz;     // Detected fundamental f1
  float centsOffset;        // Deviation from equal temperament
  float strikeRmsDb;        // Loudness of the strike in dBFS
  float measuredB;          // Estimated string stiffness B
};

class PianoCalibrator {
public:
  // Mode 0 (Recommended Fast 88-Key Split-Scale): C1, C2, C3, C4, A4, C5, C6 (7 keys = 14
strikes)
  static constexpr std::array<int, 7> kFastSplitScaleSeq = {24, 36, 48, 60, 69, 72, 84};

  // Mode 1 (Full Middle-Octave Chromatic + Multi-Octave Anchors): 16 keys = 32 strikes
  static constexpr std::array<int, 16> kFullChromaticSeq = {
    24, 36, 48, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72
  };

  void start(int sampleRate = 48000, bool fullChromaticMode = false,
             int minPlayableMidi = 21, int maxPlayableMidi = 108) noexcept {
    sr_ = sampleRate;
    fullMode_ = fullChromaticMode;
    totalKeys_ = fullChromaticMode ? 16 : 7;
    keyIdx_ = 0;
```

```
  strikeIdx_ = 1;
  state_ = State::MeasureNoise;
  noiseHops_ = 0;
  noiseEnergySum_ = 0.0f;
  prevRmsLinear_ = 1e-6f;
  prevHopDb_ = -60.0f;
  cooldownHops_ = 0;
  minObservedOvertoneRatio_ = 10.0f;
  maxObservedOvertoneRatio_ = 0.05f;
  minObservedCombRatio_ = 1.0f;

  profile_.minCalibratedMidi = std::clamp(minPlayableMidi, 21, 108);
  profile_.maxCalibratedMidi = std::clamp(maxPlayableMidi, 21, 108);
  profile_.minVelocityDb = -10.0f;
  profile_.maxVelocityDb = -50.0f;

  for (int m = 0; m < 128; ++m) {
   profile_.keyFreqHz[m] = 440.0f * std::pow(2.0f, (m - 69) / 12.0f);
   profile_.inharmonicityB[m] = 0.00035f;
   measuredKeyMask_[m] = false;
  }
 }

 // Called once per 256-sample hop by Engine analysis thread using a 4096-sample lookback
window
 bool processWindow(const float* window4096, const float* hop256,
                    CalibrationProgressEvent& outEvent) noexcept {
  float sumSq = 0.0f;
  for (int i = 0; i < 256; ++i) sumSq += hop256[i] * hop256[i];
  const float rmsLinear = std::sqrt(sumSq / 256.0f + 1e-12f);
  const float rmsDb = 20.0f * std::log10(rmsLinear);
  const float fluxRatio = rmsLinear / (prevRmsLinear_ + 1e-7f);
  const float deltaDb = rmsDb - prevHopDb_;
  prevRmsLinear_ = rmsLinear;
  prevHopDb_ = rmsDb;

  if (cooldownHops_ > 0) {
   --cooldownHops_;
   return false;
  }

  // Phase 0: Measure ambient room noise over 30 hops (~160 ms)
  if (state_ == State::MeasureNoise) {
```

```
  noiseEnergySum_ += rmsDb;
  if (++noiseHops_ >= 30) {
    profile_.ambientNoiseDb = noiseEnergySum_ / 30.0f;
    profile_.calibratedGateDb = std::clamp(profile_.ambientNoiseDb + 8.0f, -58.0f, -32.0f);
    state_ = State::WaitStrike;
    emitEvent(outEvent, 1, 0.0f, 0.0f, profile_.ambientNoiseDb, 0.0f);
    return true;
  }
  return false;
}

if (state_ == State::Done) return false;

const int targetMidi = currentTargetMidi();
const float nomFreq = 440.0f * std::pow(2.0f, (targetMidi - 69) / 12.0f);

// WaitReleaseValley: Prevents Strike 1 sustain tail from bleeding into Strike 2
if (state_ == State::WaitReleaseValley) {
  valleyFloorDb_ = std::min(valleyFloorDb_, rmsDb);
  const bool droppedEnough =
      (valleyFloorDb_ < peakRmsDb_ - 5.0f) || (rmsDb < profile_.calibratedGateDb + 2.0f);
  const bool freshAttack = (deltaDb > 4.5f && rmsDb > valleyFloorDb_ + 5.0f);
  if (droppedEnough && freshAttack) {
    state_ = State::SettleStrike;
    settleCount_ = 0;
    peakRmsDb_ = rmsDb;
    peakFlux_ = fluxRatio;
  }
  return false;
}

// Wait for initial key onset above calibrated noise gate
if (state_ == State::WaitStrike) {
  if (rmsDb > profile_.calibratedGateDb + 4.0f && (fluxRatio > 1.45f || deltaDb > 4.5f)) {
    state_ = State::SettleStrike;
    settleCount_ = 0;
    peakRmsDb_ = rmsDb;
    peakFlux_ = fluxRatio;
  }
  return false;
}

// Wait 3 hops (~16 ms) after hammer transient so 4096-sample window contains clean tonal
```

```
body
   if (state_ == State::SettleStrike) {
    peakRmsDb_ = std::max(peakRmsDb_, rmsDb);
    if (++settleCount_ >= 3) {
     const int verifyStatus =
        analyzeAndVerifyStrike(window4096, targetMidi, nomFreq, strikeIdx_, outEvent);
     if (verifyStatus < 0) {
      // -1 = Wrong key played, -2 = Whistle / clap / voice / non-piano noise rejected
      outEvent.state = verifyStatus;
      cooldownHops_ = 24;
      state_ = (strikeIdx_ == 1) ? State::WaitStrike : State::WaitReleaseValley;
      valleyFloorDb_ = peakRmsDb_;
      return true;
     }

     cooldownHops_ = 16; // ~85 ms minimum guard before valley detector arms
     if (strikeIdx_ == 1) {
      strikeIdx_ = 2;
      valleyFloorDb_ = peakRmsDb_;
      state_ = State::WaitReleaseValley;
      outEvent.state = 2; // Prompt UI for Strike 2 of 2
     } else {
      finalizeKey(targetMidi);
      if (++keyIdx_ >= totalKeys_) {
       finalizeSplitScaleProfile();
       state_ = State::Done;
       outEvent.state = 4; // All calibration keys complete
      } else {
       strikeIdx_ = 1;
       state_ = State::WaitStrike;
       outEvent.state = 3; // Key complete, advance UI to next key
      }
     }
     return true;
    }
   }
   return false;
  }

  const PianoProfile& profile() const noexcept { return profile_; }

private:
  enum class State { MeasureNoise, WaitStrike, WaitReleaseValley, SettleStrike, Done };
```

```
 int currentTargetMidi() const noexcept {
  const int idx = std::min(keyIdx_, totalKeys_ - 1);
  return fullMode_ ? kFullChromaticSeq[idx] : kFastSplitScaleSeq[idx];
 }

 float findPeakFrequency(const float* x4096, float centerHz, float rangeCents,
                         float& outMag) const noexcept {
  constexpr int kSteps = 41;
  float bestFreq = centerHz;
  float bestMagSq = -1.0f;
  for (int i = 0; i < kSteps; ++i) {
   const float c = -rangeCents + (2.0f * rangeCents * i) / (kSteps - 1);
   const float candHz = centerHz * std::pow(2.0f, c / 1200.0f);
   const float magSq = Goertzel::magnitudeSquared(x4096, 4096, candHz, sr_);
   if (magSq > bestMagSq) {
    bestMagSq = magSq;
    bestFreq = candHz;
   }
  }
  outMag = std::sqrt(std::max(0.0f, bestMagSq));
  return bestFreq;
 }

 // Returns 0 on valid piano strike, -1 on wrong key, -2 on whistle/clap/voice/non-piano sound
 int analyzeAndVerifyStrike(const float* x4096, int midi, float nomFreq, int strikeNum,
                            CalibrationProgressEvent& ev) noexcept {
  float mag1 = 0.0f;
  const float f1 = findPeakFrequency(x4096, nomFreq, 60.0f, mag1);

  // 1. Wrong-Key Check: Verify adjacent semitones (+/-1, +/-2) do not have stronger
fundamentals
  for (int offset : {-2, -1, 1, 2}) {
   float otherMag = 0.0f;
   const float otherNom = 440.0f * std::pow(2.0f, (midi + offset - 69) / 12.0f);
   findPeakFrequency(x4096, otherNom, 35.0f, otherMag);
   if (otherMag > mag1 * 1.35f) {
    emitEvent(ev, strikeNum, otherNom, 0.0f, peakRmsDb_, 0.0f);
    return -1; // Wrong key struck by user
   }
  }

  // 2. Measure partials h = 2..6 for Fletcher inharmonicity B, Comb HNR, and Timbre Profile
```

```
  float bSum = 0.0f;
  int bCount = 0;
  float overtoneEnergySq = 0.0f;
  float totalCombEnergySq = mag1 * mag1;

  for (int h = 2; h <= 6; ++h) {
   float magH = 0.0f;
   const float expectedH = h * f1 * std::sqrt(1.0f + 0.00035f * h * h);
   const float fh = findPeakFrequency(x4096, expectedH, 45.0f, magH);
   if (h <= 4) overtoneEnergySq += magH * magH;
   totalCombEnergySq += magH * magH;

   const float ratioSq = (fh / (h * f1)) * (fh / (h * f1));
   if (ratioSq > 0.999f && ratioSq < 1.09f) {
    const float bEst = std::max(0.0f, (ratioSq - 1.0f) / static_cast<float>(h * h));
    bSum += bEst;
    ++bCount;
   }
   if (strikeNum == 2 && mag1 > 1e-5f) {
    const float w = std::clamp(magH / mag1, 0.02f, 2.5f);
    profile_.partialWeights[h - 1] = 0.70f * profile_.partialWeights[h - 1] + 0.30f * w;
   }
  }

  // Compute total 4096-sample window energy to verify In-Comb Harmonic Concentration
  float windowEnergy = 0.0f;
  for (int i = 0; i < 4096; ++i) windowEnergy += x4096[i] * x4096[i];
  const float combRatio = (totalCombEnergySq / 2048.0f) / (windowEnergy + 1e-9f);

  // Reject pure-sine whistles (overtoneRatio < 0.03) AND broadband claps/knocks (combRatio
< 0.14)
  const float overtoneRatio = overtoneEnergySq / (mag1 * mag1 + 1e-9f);
  if (overtoneRatio < 0.03f || bCount < 1 || combRatio < 0.14f) {
   emitEvent(ev, strikeNum, f1, 0.0f, peakRmsDb_, 0.0f);
   return -2; // Rejected: non-piano sound (whistle, clap, voice, or knock)
  }

  // Record observed middle/treble piano acoustic fingerprint bounds
  if (midi >= 48 && midi <= 84) {
   minObservedOvertoneRatio_ = std::min(minObservedOvertoneRatio_, overtoneRatio);
   maxObservedOvertoneRatio_ = std::max(maxObservedOvertoneRatio_, overtoneRatio);
   minObservedCombRatio_ = std::min(minObservedCombRatio_, combRatio);
  }
```

```
  const float bMeasured = bSum / static_cast<float>(bCount);
  const float cents = 1200.0f * std::log2(f1 / nomFreq);

  if (strikeNum == 1) {
    f1Strike1_ = f1;
    bStrike1_ = bMeasured;
    rmsStrike1_ = peakRmsDb_;
    fluxStrike1_ = peakFlux_;
  } else {
    f1Strike2_ = f1;
    bStrike2_ = bMeasured;
    rmsStrike2_ = peakRmsDb_;
    fluxStrike2_ = peakFlux_;
  }

  emitEvent(ev, strikeNum, f1, cents, peakRmsDb_, bMeasured);
  return 0;
 }

 void finalizeKey(int midi) noexcept {
  profile_.keyFreqHz[midi] = 0.5f * (f1Strike1_ + f1Strike2_);
  profile_.inharmonicityB[midi] = 0.35f * bStrike1_ + 0.65f * bStrike2_;
  measuredKeyMask_[midi] = true;

  const float loRms = std::min(rmsStrike1_, rmsStrike2_);
  const float hiRms = std::max(rmsStrike1_, rmsStrike2_);
  profile_.minVelocityDb = std::min(profile_.minVelocityDb, loRms);
  profile_.maxVelocityDb = std::max(profile_.maxVelocityDb, hiRms);

  const float loFlux = std::min(fluxStrike1_, fluxStrike2_);
  profile_.calibratedFluxRatio = std::clamp(loFlux * 0.85f, 1.35f, 2.40f);
 }

 // Fits two separate exponential curves (Bass Bridge C1..C3 and Treble Bridge C3..C6),
 // synthesizes the 88-key Railsback stretched tuning table, and locks the Acoustic Fingerprint
bounds
 void finalizeSplitScaleProfile() noexcept {
  if (measuredKeyMask_[69]) {
    profile_.a4ReferenceHz = profile_.keyFreqHz[69];
  } else if (measuredKeyMask_[60]) {
    const float et60 = 440.0f * std::pow(2.0f, (60 - 69) / 12.0f);
    profile_.a4ReferenceHz = 440.0f * (profile_.keyFreqHz[60] / et60);
```

```
}

profile_.meanInharmonicityB = profile_.inharmonicityB[60];

// Lock in the calibrated Acoustic Fingerprint filter bounds with safety margins
profile_.minOvertoneRatio = std::clamp(minObservedOvertoneRatio_ * 0.65f, 0.04f, 0.25f);
profile_.maxOvertoneRatio = std::clamp(maxObservedOvertoneRatio_ * 1.45f, 1.50f, 6.00f);
profile_.minCombEnergyRatio = std::clamp(minObservedCombRatio_ * 0.70f, 0.15f, 0.40f);
profile_.minTimbreSimilarity = 0.76f;
profile_.calibratedPitchWindowCents = 22.0f;

const float b24 = std::max(1e-5f, profile_.inharmonicityB[24]);
const float b36 = std::max(1e-5f, profile_.inharmonicityB[36]);
const float bassSlope = std::log(b24 / b36) / 12.0f;
profile_.bassInharmonicityB0 = b36;
profile_.bassSlope = bassSlope;

const float b60 = std::max(1e-5f, profile_.inharmonicityB[60]);
const float bTop = measuredKeyMask_[84] ? std::max(1e-5f, profile_.inharmonicityB[84])
                                        : std::max(1e-5f, profile_.inharmonicityB[72]);
const float spanSemis = measuredKeyMask_[84] ? 24.0f : 12.0f;
const float trebleSlope = std::clamp(std::log(bTop / b60) / spanSemis, 0.02f, 0.065f);
profile_.trebleInharmonicityB0 = b60;
profile_.trebleSlope = trebleSlope;

const float bRef69 = b60 * std::exp(trebleSlope * (69.0f - 60.0f));
for (int m = 21; m <= 108; ++m) {
 if (!measuredKeyMask_[m]) {
  if (m < 48) {
   profile_.inharmonicityB[m] =
     std::clamp(b36 * std::exp(bassSlope * (36.0f - m)), 0.00005f, 0.015f);
  } else {
   profile_.inharmonicityB[m] =
     std::clamp(b60 * std::exp(trebleSlope * (m - 60.0f)), 0.00005f, 0.025f);
  }

  const float hTune = (m < 48) ? 3.0f : 2.0f;
  const float stretchFactor =
    std::sqrt((1.0f + profile_.inharmonicityB[m] * hTune * hTune) /
          (1.0f + bRef69 * hTune * hTune));
  const float direction = (m < 60) ? (1.0f / stretchFactor) : stretchFactor;
  profile_.keyFreqHz[m] =
    profile_.a4ReferenceHz * std::pow(2.0f, (m - 69) / 12.0f) * direction;
```

```
   }
  }
  profile_.isCalibrated = true;
}

void emitEvent(CalibrationProgressEvent& ev, int strikeNum, float freq,
               float cents, float rmsDb, float bVal) const noexcept {
  ev.currentMidi = currentTargetMidi();
  ev.keyIndex = keyIdx_;
  ev.totalKeys = totalKeys_;
  ev.strikeNumber = strikeNum;
  ev.state = (state_ == State::MeasureNoise) ? 1 : strikeNum;
  ev.measuredFreqHz = freq;
  ev.centsOffset = cents;
  ev.strikeRmsDb = rmsDb;
  ev.measuredB = bVal;
}

int sr_ = 48000;
bool fullMode_ = false;
int totalKeys_ = 7;
int keyIdx_ = 0;
int strikeIdx_ = 1;
State state_ = State::MeasureNoise;
int noiseHops_ = 0;
float noiseEnergySum_ = 0.0f;
float prevRmsLinear_ = 1e-6f;
float prevHopDb_ = -60.0f;
float valleyFloorDb_ = -60.0f;
int settleCount_ = 0;
int cooldownHops_ = 0;
float peakRmsDb_ = -60.0f;
float peakFlux_ = 1.0f;

float minObservedOvertoneRatio_ = 10.0f;
float maxObservedOvertoneRatio_ = 0.05f;
float minObservedCombRatio_ = 1.0f;

float f1Strike1_ = 0.0f, f1Strike2_ = 0.0f;
float bStrike1_ = 0.0f, bStrike2_ = 0.0f;
float rmsStrike1_ = -48.0f, rmsStrike2_ = -20.0f;
float fluxStrike1_ = 1.8f, fluxStrike2_ = 2.2f;
std::array<bool, 128> measuredKeyMask_{};
```

```
  PianoProfile profile_{};
};

} // namespace piano
```

v1.1 Upgrade 4: Guided React Native Calibration UI (src/CalibrationScreen.tsx)

In **v1.1**, src/CalibrationScreen.tsx guides the user through the two-strike calibration, displays the learned acoustic range (minVelocityDb to maxVelocityDb, measuredB, and centsOffset), and informs the user that all sounds outside the calibrated piano fingerprint (whistles, speech, claps, background noise, and out-of-range frequencies) are locked out:

TypeScript

```
// src/CalibrationScreen.tsx (New in v1.1)
import React, {useState, useEffect} from 'react';
import {View, Text, TouchableOpacity, StyleSheet} from 'react-native';

export type CalibrationStepEvent = {
  currentMidi: number;
  keyIndex: number;
  totalKeys: number;
  strikeNumber: number; // 1 or 2
  state: number;       // -2=NonPianoNoiseRejected, -1=WrongKey, 0=Noise, 1=WaitStrike1, 2=WaitStrike2, 3=KeyDone, 4=AllDone
  measuredFreqHz: number;
  centsOffset: number;
  strikeRmsDb: number;
  measuredB: number;
};

const NOTE_NAMES: Record<number, {label: string; hint: string}> = {
  24: {label: 'C1 (Deep Bass C)', hint: 'Lowest C on the left side (calibrates copper-wound bass bridge)'},
  36: {label: 'C2 (Bass C)', hint: 'Two octaves below Middle C (calibrates octave 2 minimum stiffness)'},
  48: {label: 'C3 (Bridge Break C)', hint: 'One octave below Middle C (calibrates bass-to-treble transition)'},
  60: {label: 'C4 (Middle C)', hint: 'Center white key just left of the two black keys'},
```

```
  61: {label: 'C#4', hint: 'First black key to the right of Middle C'},
  62: {label: 'D4', hint: 'White key between C#4 and D#4'},
  63: {label: 'D#4', hint: 'Second black key in the two-black-key group'},
  64: {label: 'E4', hint: 'White key immediately right of D#4'},
  65: {label: 'F4', hint: 'White key just left of the three black keys'},
  66: {label: 'F#4', hint: 'First black key in the three-black-key group'},
  67: {label: 'G4', hint: 'White key between F#4 and G#4'},
  68: {label: 'G#4', hint: 'Middle black key in the three-black-key group'},
  69: {label: 'A4 (440 Hz Reference)', hint: 'Five white keys above Middle C (sets global concert
pitch)'},
  70: {label: 'A#4', hint: 'Third black key in the three-black-key group'},
  71: {label: 'B4', hint: 'White key immediately right of A#4'},
  72: {label: 'C5 (Treble C)', hint: 'One octave above Middle C (calibrates upper treble stretch)'},
  84: {label: 'C6 (High Treble C)', hint: 'Two octaves above Middle C (calibrates high treble
stiffness)'},
};

export function CalibrationScreen({
  onStartCalibration,
  onComplete,
  latestEvent,
}: {
  onStartCalibration: (fullChromaticMode: boolean) => void;
  onComplete: () => void;
  latestEvent?: CalibrationStepEvent;
}) {
  const [started, setStarted] = useState(false);
  const [fullMode, setFullMode] = useState(false);

  const midi = latestEvent?.currentMidi ?? 24;
  const noteMeta = NOTE_NAMES[midi] ?? {label: `MIDI ${midi}`, hint: ''};
  const strikeNum = latestEvent?.state === 2 ? 2 : 1;
  const isAllDone = latestEvent?.state === 4;

  const classifyPianoType = (b: number) => {
    if (b < 0.00012) return 'Digital / Sampled Piano (Zero String Stretch)';
    if (b < 0.00048) return 'Grand / Large Acoustic Piano (Low Inharmonicity)';
    return 'Compact Upright / Spinet Piano (Split-Scale Stiffness Calibrated)';
  };

  useEffect(() => {
    if (isAllDone) {
      onComplete();
```

```
  }
 }, [isAllDone, onComplete]);

 return (
  <View style={styles.container}>
   <Text style={styles.title}>Split-Scale Piano Profile & Calibration (v1.1)</Text>
   <Text style={styles.subtitle}>
    Press each prompted key <Text style={styles.bold}>2 times</Text> (once gently, once
firmly).
    Once calibrated, our v1.1 Acoustic Fingerprint Gate locks onto your piano's exact frequency
    grid, harmonic timbre, and dynamic range—automatically blocking all outside noises
(whistles,
    talking, claps, coughs, and background audio).
   </Text>

   {!started ? (
    <View style={styles.modeContainer}>
     <TouchableOpacity
      style={styles.primaryBtn}
      onPress={() => {
       setFullMode(false);
       setStarted(true);
       onStartCalibration(false);
      }}>
      <Text style={styles.primaryBtnText}>
       Fast 88-Key Split-Scale (Recommended • 7 Keys: C1–C6 + A4)
      </Text>
     </TouchableOpacity>

     <TouchableOpacity
      style={styles.secondaryBtn}
      onPress={() => {
       setFullMode(true);
       setStarted(true);
       onStartCalibration(true);
      }}>
      <Text style={styles.secondaryBtnText}>
       Full Chromatic Middle Octave + Bass/Treble Anchors (16 Keys)
      </Text>
     </TouchableOpacity>
    </View>
   ) : (
    <View style={styles.card}>
```

```
<Text style={styles.stepCounter}>
  Key {(latestEvent?.keyIndex ?? 0) + 1} of {latestEvent?.totalKeys ?? (fullMode ? 16 : 7)}
</Text>

<Text style={styles.targetKey}>{noteMeta.label}</Text>
<Text style={styles.keyHint}>{noteMeta.hint}</Text>

{latestEvent?.state === -1 && (
  <View style={styles.warningBanner}>
    <Text style={styles.warningText}>
      Wrong key detected! Please press {noteMeta.label} only.
    </Text>
  </View>
)}

{latestEvent?.state === -2 && (
  <View style={styles.warningBanner}>
    <Text style={styles.warningText}>
      Out-of-calibration sound (whistle, speech, clap, or room noise) blocked. Please
      strike the piano key.
    </Text>
  </View>
)}

<View style={styles.strikeRow}>
  <View style={[styles.strikeBadge, strikeNum === 1 && styles.strikeBadgeActive]}>
    <Text style={styles.strikeTitle}>Strike 1 of 2 (Soft / Normal)</Text>
    <Text style={styles.strikeDesc}>
      Press {noteMeta.label.split(' ')[0]} gently to capture your minimum dynamic threshold
      and fundamental pitch.
    </Text>
  </View>

  <View style={[styles.strikeBadge, strikeNum === 2 && styles.strikeBadgeActive]}>
    <Text style={styles.strikeTitle}>Strike 2 of 2 (Firm)</Text>
    <Text style={styles.strikeDesc}>
      Wait a brief moment for the first note to settle, then press{' '}
      {noteMeta.label.split(' ')[0]} firmly to lock in your piano's harmonic timbre
      template and maximum dynamic range.
    </Text>
  </View>
</View>
```

```
        {latestEvent && latestEvent.measuredFreqHz > 0 && latestEvent.state > 0 && (
          <View style={styles.telemetryBox}>
            <Text style={styles.telemetryText}>
              Measured Pitch: {latestEvent.measuredFreqHz.toFixed(2)} Hz (
              {latestEvent.centsOffset >= 0 ? '+' : ''}
              {latestEvent.centsOffset.toFixed(1)} ¢)
            </Text>
            <Text style={styles.telemetryText}>
              Strike Level: {latestEvent.strikeRmsDb.toFixed(1)} dBFS | Stiffness B:{' '}
              {latestEvent.measuredB.toFixed(5)}
            </Text>
            <Text style={styles.profileBadge}>
              Detected Profile: {classifyPianoType(latestEvent.measuredB)} (Outside Noises Locked
Out)
            </Text>
          </View>
        )}
      </View>
    )}
  </View>
 );
}

const styles = StyleSheet.create({
 container: {flex: 1, padding: 24, backgroundColor: '#121316', justifyContent: 'center'},
 title: {fontSize: 24, fontWeight: '700', color: '#FFFFFF', marginBottom: 8},
 subtitle: {fontSize: 14, color: '#A0A6B2', lineHeight: 20, marginBottom: 24},
 bold: {color: '#FFFFFF', fontWeight: '700'},
 modeContainer: {flexDirection: 'column', gap: 12},
 primaryBtn: {backgroundColor: '#1A73E8', paddingVertical: 16, paddingHorizontal: 14,
borderRadius: 10, alignItems: 'center'},
 primaryBtnText: {color: '#FFFFFF', fontSize: 15, fontWeight: '600', textAlign: 'center'},
 secondaryBtn: {backgroundColor: '#1E2025', paddingVertical: 14, paddingHorizontal: 14,
borderRadius: 10, alignItems: 'center', borderWidth: 1, borderColor: '#3C4043'},
 secondaryBtnText: {color: '#8AB4F8', fontSize: 14, fontWeight: '600', textAlign: 'center'},
 card: {backgroundColor: '#1E2025', borderRadius: 14, padding: 20, borderWidth: 1,
borderColor: '#2E323B'},
 stepCounter: {fontSize: 12, fontWeight: '600', color: '#8AB4F8', textTransform: 'uppercase'},
 targetKey: {fontSize: 30, fontWeight: '700', color: '#FFFFFF', marginTop: 6},
 keyHint: {fontSize: 13, color: '#A0A6B2', marginTop: 4, marginBottom: 16},
 warningBanner: {backgroundColor: '#371C1B', borderColor: '#F28B82', borderWidth: 1,
borderRadius: 8, padding: 10, marginBottom: 14},
 warningText: {color: '#F28B82', fontSize: 13, fontWeight: '600'},
```

```
  strikeRow: {flexDirection: 'column', gap: 12},
  strikeBadge: {padding: 14, borderRadius: 10, backgroundColor: '#141619', borderWidth: 1,
borderColor: '#2B2F36'},
  strikeBadgeActive: {borderColor: '#1A73E8', backgroundColor: '#17223B'},
  strikeTitle: {fontSize: 15, fontWeight: '700', color: '#FFFFFF', marginBottom: 4},
  strikeDesc: {fontSize: 13, color: '#C4C9D4', lineHeight: 18},
  telemetryBox: {marginTop: 18, paddingTop: 14, borderTopWidth: 1, borderTopColor: '#2E323B'},
  telemetryText: {fontSize: 12, color: '#A0A6B2', fontFamily: 'monospace', marginBottom: 4},
  profileBadge: {fontSize: 13, fontWeight: '600', color: '#81C995', marginTop: 4},
});
```

Acoustic Calibration and Deployment Recommendations

While the portable C++ core achieves 100% single-note accuracy (88/88 keys) and 30 µs mean hop execution on synthetic grand-piano benchmarks, deploying to physical mobile hardware in reverberant rooms requires targeted calibration of shared/Cascade.h[1].

Real-Device Telemetry Verification

Because the baseline v1.0 benchmarks were executed on an x86 desktop harness rather than physical phones, engineers should immediately poll getEngineStats() (avgUs, maxUs, and lateHops) on target Android and iOS handsets during live acoustic performance[1]. Monitoring these telemetry values verifies that worst-case execution (maxUs) remains well below the 5333 µs hop deadline and that lateHops remains zero even under mobile thermal throttling or background OS activity[1]. Engineers must also account for the fact that event timestamps (tMs) measure stream time elapsed since startDetector() and do not include OS or hardware microphone input buffer latency, which must be calibrated as a constant offset when scoring rhythmic timing accuracy[1].

Resolving Octave Ambiguity in Expected-Notes Mode

A known physical ambiguity occurs in v1.0 setExpectedNotes mode whenever the expected note set contains both a pitch p and its octave above $p + 12$ (for example, C_3 and C_4)[1]. Playing the lower note (C_3) alone will also fire a false-positive noteOn for the upper note (C_4) because C_3's second partial ($2f_0$) coincides directly with C_4's fundamental frequency, and C_3's fourth partial ($4f_0$) coincides with C_4's second partial[1]. In v1.1, Cascade.h resolves this ambiguity using the calibrated partialWeights envelope: when the lower pitch p is active, confirming the upper pitch $p + 12$ requires observing an energy rise at $2f_0$ and $6f_0$ (the

first and third odd partials of the upper note) that significantly exceeds the smooth $1/m$ spectral decay envelope ($g(\tau, m) = (f_s/\tau + \alpha)/(mf_s/\tau + \beta)$) established by the lower note's odd partials ($f_0, 3f_0, 5f_0$)[1].

Calibrating for Small Upright Pianos and Hybrid Neural Filtering

Short upright pianos exhibit up to 4× higher string stiffness (B) than concert grands, stretching upper partials enough to reduce v1.0 single-note bass detection from 27/27 to 24/27 and inducing ghost notes in the experimental chord stage[1]. Using the **v1.1 Split-Scale Calibration** curve in Calibration.h to fit separate bass and treble inharmonicity slopes $B(p)$ across the bridge transition at C_3 restores full bass capture on small uprights and spinets[1]. Furthermore, if future iterations of the application require free-form polyphonic chord transcription without ghost notes, developers should avoid loading heavy ONNX Runtime or TFLite models on the audio thread[20]. Instead, keeping the C++ Gated Cascade as the ultra-fast front-end feature extractor and feeding its candidate partial energies into a compact, compile-time **RTNeural** multilayer perceptron or 16-unit LSTM (RTNeural::ModelT) provides an ideal hybrid extension[48]. Because RTNeural utilizes compile-time template unrolling and SIMD backends (XSIMD and Eigen) with zero runtime heap allocations, it executes 2–10× faster than ONNX Runtime inside hard real-time C++ audio threads, suppressing harmonic ghost notes while preserving sub-10 ms responsiveness[47].

Works cited

1. README.md
2. React Native Development Company | TantraDev | TantraDev, https://www.tantradev.com/technologies/react-native
3. Top React Native Interview Questions & Answers 2026, https://www.techinterview.net/questions/react-native-interview-questions
4. AVAudioSession.h - GitHub, https://github.com/theos/sdks/blob/master/iPhoneOS10.3.sdk/System/Library/Frameworks/AVFoundation.framework/Frameworks/AVFAudio.framework/Headers/AVAudioSession.h
5. High Performance Game Audio with Oboe - Android Developers Blog, https://android-developers.googleblog.com/2021/03/high-performance-game-audio-with-oboe.html
6. Recently Active 'avaudiosession' Questions - Page 10 - Stack Overflow, https://stackoverflow.com/questions/tagged/avaudiosession?tab=Active&page=10
7. 2 Pitch analysis | Automatic Music Transcription - Rand Asswad, https://rand-asswad.xyz/muallef/2_pitch.html
8. I R I S A, https://perso.telecom-paristech.fr/rbadeau/assets/irisa-09.pdf

9. Optimizing a lock-free ring buffer - Hacker News, https://news.ycombinator.com/item?id=47501875
10. Low Level Fundamentals: Lock Free Programming and Atomics, https://yash30201.substack.com/p/low-level-fundamentals-lock-free
11. Lock-Free SPSC Ring Buffers in Low-Latency ... - HackerPrep Blog, https://hackerprep.io/blog/lock-free-spsc-ring-buffers-in-low-latency-trading-interviews-c-what-interviewers-look-for-and-how-to-reason-about-memory-ordering
12. 无锁队列SPSC Queue - sinkinben - 博客园, https://www.cnblogs.com/sinkinben/p/17949761/spsc-queue
13. veetihietasalo/MiniHFT: Ultra-low latency market making engine in C, https://github.com/veetihietasalo/MiniHFT
14. cppweeklynews/posts/202.md at dev - GitHub, https://github.com/wanghenshui/cppweeklynews/blob/dev/posts/202.md
15. 从零实现SPSC 无锁队列原创 - CSDN博客, https://blog.csdn.net/cookies_s_/article/details/160229302
16. The Audio Processing Dog House - objc.io, https://www.objc.io/issue-24/audio-dog-house.html
17. Keep an eye on CycFi's "Hz" pitch detection project; "Q" DSP library, https://forum.pjrc.com/index.php?threads/suggestion-audio-library-keep-an-eye-on-cycfis-hz-pitch-detection-project-q-dsp-library.75957/
18. PYIN: A fundamental frequency estimator using probabilistic, https://www.researchgate.net/publication/269295422_PYIN_A_fundamental_frequency_estimator_using_probabilistic_threshold_distributions
19. Alexandre R.J. François, “Resonate: Efficient Low Latency Spectral, https://alexandrefrancois.org/assets/publications/FrancoisARJ-ICMC2025.pdf
20. Accurate and Lightweight Instrument-Agnostic Music Transcription, https://arxiv.org/pdf/2609.04640
21. translating Machine Learning Pipelines to Android - Arm Developer, https://developer.arm.com/community/arm-community-blogs/b/ai-blog/posts/translating-machine-learning-pipelines-to-android
22. A Lightweight Instrument-Agnostic Model for Polyphonic Note, https://www.researchgate.net/publication/360792948_A_Lightweight_Instrument-Agnostic_Model_for_Polyphonic_Note_Transcription_and_Multipitch_Estimation
23. CrispStrobe/CrispASR: C++ ggml runtime hub for multilingual ASR, https://github.com/CrispStrobe/CrispASR
24. Daily Papers - Hugging Face, https://huggingface.co/papers?q=loudness%20control
25. End-to-End Deep Learning for Audio Classification - Espace ETS, https://espace.etsmtl.ca/id/eprint/2895/1/ABDOLI_Sajjad.pdf
26. Vocal Interactivity in-and-between Humans, Animals and Robots, https://vihar-2024.vihar.org/assets/VIHAR_2024_proceedings.pdf
27. Elegant Manual - TuneLab World, https://www.tunelab-world.com/TuneLabPro5.1Manual.pdf
28. Multiple Fundamental Frequency Estimation by Summing Harmonic,

https://www.semanticscholar.org/paper/Multiple-Fundamental-Frequency-Estimation-by-Klapuri/a54abe400902619e851c2f86f79018f708f501ef
29. Multiple Fundamental Frequency Estimation by Summing Harmonic, https://archives.ismir.net/ismir2006/paper/000125.pdf
30. Processing Music Signals Using Audio Decomposition Techniques, https://audiolabs-erlangen.de/content/05_fau/professor/00_mueller/01_group/2016_Driedger_AudioDecomposition_PhD-Thesis.pdf
31. Exploring Polyphonic Piano Transcription as an Inverse Problem, https://epub.jku.at/download/pdf/8025589.pdf
32. Inharmonicity - TuneLab World, https://www.tunelab-world.com/tlpihcalc.html
33. What's New - TuneLab World, https://tunelab-world.com/atl-whatsnew.html
34. Measure Inharmonicity Button - TuneLab World, https://www.tunelab-world.com/mtuner/MeasureInharmonicityButton.html
35. Signal to Noise Ratio Formula - GeeksforGeeks, https://www.geeksforgeeks.org/physics/signal-to-noise-ratio-formula/
36. TuneLab Piano Tuner, https://www.tunelab-world.com/TuneLabAndroidManual-2.4.pdf
37. TuneLab Piano Tuner, https://tunelab-world.com/TuneLab-iOS-Manual-4.6.pdf
38. Real-Time Pitch Estimation in Music Using Frequency Convolution, https://www.mdpi.com/2079-9292/15/17/3961
39. Wavetable signal to noise ratio - EarLevel Engineering, https://www.earlevel.com/main/2018/09/22/wavetable-signal-to-noise-ratio/
40. Improvements in estimating the harmonics-to-noise ratio of the voice, https://pubmed.ncbi.nlm.nih.gov/7987428/
41. Software-Based Extraction of Objective Parameters from Music, https://vtechworks.lib.vt.edu/bitstream/handle/10919/71624/384_1.pdf
42. Pitch and Harmonic to Noise Ratio Estimation, https://www.audiolabs-erlangen.de/content/05_fau/professor/00_mueller/02_teaching/2017w-apl/LabCourse_Pitch.pdf
43. Instructions & Support - PianoMeter, https://pianometer.com/support/
44. A question about Piano Meter's inharmonicity graph, https://forum.pianoworld.com/ubbthreads.php/topics/3560337/a-question-about-piano-meters-inharmonicity-graph.html
45. PianoMeter – Piano Tuner | Professional Piano Tuning App for, https://pianometer.com/
46. Tuning with TuneLab – Class Notes | Midwest PTG, https://midwestptg.com/wp-content/uploads/2014/09/ClassNotes.pdf
47. RTNeural real-time neural inference UGen - scsynth, https://scsynth.org/t/rtneural-real-time-neural-inference-ugen/10074
48. jatinchowdhury18/RTNeural: Real-time neural network inferencing, https://github.com/jatinchowdhury18/RTNeural
49. Add support for NAM files · Issue #143 · jatinchowdhury18/RTNeural, https://github.com/jatinchowdhury18/RTNeural/issues/143
50. RTNeural: Fast Neural Inferencing for Real-Time Systems - arXiv, https://arxiv.org/html/2106.03037v1